



TALENT FLOW: REAL-TIME REMOTE TECHNICAL INTERVIEW & ASSESSMENT PLATFORM

A MINI PROJECT REPORT

Submitted by

AROMAL ANIL

MLM25MCA-2024

To

the APJ Abdul Kalam Technological University in
Partial fulfillment of the requirements for the
award of the degree

of

MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. ANJALY SAJEEV

(Assistant Professor, Department of Computer Applications)



**DEPARTMENT OF COMPUTER APPLICATIONS
MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR**

(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026



MANGALAM COLLEGE OF ENGINEERING
Accredited by NAAC & ISO 9001:2000 Certified Institution
DEPARTMENT OF COMPUTER APPLICATIONS

VISION

To become a centre of excellence in computer applications, competent in the global ecosystem with technical knowledge, innovation with a sense of social commitment.

MISSION

- To serve with state of the art education, foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PEO I : Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PEO II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PEO III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and social values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

- **PSO I:** Apply advanced technologies through innovations to enhance the efficiency of design and development.
- **PSO II:** Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO-PSO-SDG

1. MAPPING WITH PROGRAM OUTCOMES (POs):

SL.NO	POs ADDRESSED	RELEVANCE TO PROJECT
1	PO1	Applies programming, database, and coding concepts to develop the Talent Flow platform.
2	PO2	Analyzes challenges in remote technical interviews, coding, assessment, and monitoring.
3	PO3	Develops an integrated platform for online interviews, collaborative coding, and candidate assessment.
4	PO4	Uses modern technologies such as React, Node.js, WebRTC, Socket.io, and AI services.
5	PO7	Applies security and ethical practices to protect candidate and interview information.
6	PO8	Applies security and ethical practices to protect candidate and interview information.

LIST OF PROGRAM OUTCOMES (POs):

PO1 (Foundation Knowledge): Apply knowledge of programming logic, database concepts and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyze problems in traditional complaint reporting with focus on user requirements.

PO3 (Development of Solutions): Design and develop an innovative online complaint reporting solution for efficient complaint management and tracking.

PO4 (Modern Tool Usage): Select and apply modern computational tools and technologies such as PHP, MySQL, JavaScript and NLP for system development.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or team member during the planning, development and testing of the project.

PO6 (Project Management and Finance): Apply project management principles such as planning, scheduling, resource management and work breakdown structure.

PO7 (Ethics): Follow professional ethics and cybersecurity practices to protect user information, complaint data and digital evidence from unauthorized access.

PO8 (Life-long learning): Develop change management skills and continuously learn and adapt to contemporary technologies and methods for improving the complaint reporting system.

2. MAPPING WITH PROGRAM SPECIFIC OUTCOMES (PSOs):

LIST OF PROGRAM SPECIFIC OUTCOMES (PSOs):

SL.NO	PSOs ADDRESSED	RELEVANCE TO PROJECT
1	PSO1 – Apply advanced technologies through innovations to enhance the efficiency of design development	Advanced technologies such as WebRTC, WebSockets and AI-based code evaluation were applied to design an efficient real-time interview platform.
2	PSO2 – Apply the principles of computing to analyze, design and implement sustainable solutions for real-world challenges	Computing principles of data modelling, networking, security and software design were applied to build a sustainable solution for remote technical hiring.

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

3. MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG NO	SDGs ADDRESSED	RELEVANCE TO PROJECT
SDG 8	Decent Work and Economic Growth	The platform makes recruitment faster, fairer and better documented, supporting productive employment and efficient hiring.
SDG 9	Industry, Innovation and Infrastructure	The project applies real-time web technologies and artificial intelligence to create an innovative digital infrastructure for technical assessment.
SDG 10	Reduced Inequalities	Remote, browser-based interviews give candidates from any location an equal opportunity to be assessed, with consistent AI-supported evaluation.

SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG 1 – No Poverty: End poverty in all its forms everywhere.

SDG 2 – Zero Hunger: End hunger, achieve food security and improved nutrition, and promote sustainable agriculture.

SDG 3 – Good Health and Well-Being: Ensure healthy lives and promote well-being for all at all ages.

SDG 4 – Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 – Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 – Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation for all.

SDG 7 – Affordable and Clean Energy: Ensure access to affordable, reliable, sustainable and modern energy for all.

SDG 8 – Decent Work and Economic Growth: Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.

SDG 9 – Industry, Innovation and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialisation and foster innovation.

SDG 10 – Reduced Inequalities: Reduce inequality within and among countries.

SDG 11 – Sustainable Cities and Communities: Make cities and human settlements inclusive, safe, resilient and sustainable.

SDG 12 – Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 – Climate Action: Take urgent action to combat climate change and its impacts.

SDG 14 – Life Below Water: Conserve and sustainably use the oceans, seas and marine resources.

SDG 15 – Life on Land: Protect, restore and promote sustainable use of terrestrial ecosystems, manage forests sustainably, combat desertification and halt biodiversity loss.

SDG 16 – Peace, Justice and Strong Institutions: Promote peaceful and inclusive societies, provide access to justice for all and build effective, accountable and inclusive institutions.

SDG 17 – Partnerships for the Goals: Strengthen the means of implementation and revitalise the global partnership for sustainable development.

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
DEPARTMENT OF COMPUTER APPLICATIONS
OCTOBER 2026



DECLARATION

I hereby certify that the work presented in the project entitled “TALENT FLOW: Real-Time Remote Technical Interview & Assessment Platform” submitted to the DEPARTMENT OF COMPUTER APPLICATIONS is an authentic record of my own work carried out under the supervision of Ms. Anjaly Sajeev, Assistant Professor. This work has not been submitted to any other institution or university for the award of any other degree. The report has been checked for plagiarism by the college and the similarity index is within the permissible limit set by the college.

Name & Signature of Student

AROMAL ANIL

Date:

Place:

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
DEPARTMENT OF COMPUTER APPLICATIONS
OCTOBER 2026



CERTIFICATE

*Certified that the report entitled “**Talent Flow: Real-Time Remote Technical Interview & Assessment Platform**” submitted by **AROMAL ANIL (MLM25MCA-2024)** to the **APJ Abdul Kalam Technological University** in partial fulfillment of the requirements for the award of **the Degree of Master of Computer Applications** is a bonafide record of the **MINI PROJECT** work carried out by the student under our guidance and supervision during the academic year 2025-27.*

PROJECT GUIDE

Ms. Anjaly Sajeev

Assistant Professor

Department of Computer Applications

PROJECT COORDINATOR

Ms. Athira R Kurup

Assistant Professor

Department of Computer Applications

HEAD OF THE DEPARTMENT

Ms. Divya S B

Associate Professor

Department of Computer Applications

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic “*Talent Flow: Real-Time Remote Technical Interview & Assessment Platform*”.

I express my sincere thanks to **Dr. Vinodh P Vijayan**, Principal, Mangalam College of Engineering, for providing the facilities and support to complete my Project successfully.

I thank and express my sincere gratitude to **Ms. Divya S B**, HOD, Department of Computer Applications, Mangalam College of Engineering, for her invaluable help and support, which helped me a lot in successfully completing this Project work.

I express my gratitude to my Internal Guide, **Ms. Anjaly Sajeev**, Assistant Professor, Department of Computer Applications, for her valuable suggestions, guidance, and encouragement, which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculties, especially Project Coordinator, **Ms. Athira R Kurup**, Department of Computer Applications, Mangalam College of Engineering, who gave me the necessary guidance, permission, and resources required to complete the project presentation and report.

Finally, I would like to express my heartfelt thanks to my parents for their continuous support, encouragement, and motivation throughout the completion of this Project.

AROMAL ANIL

MLM25MCA-2024

ABSTRACT

Technical interviews are one of the most important and most difficult stages of recruitment, and their conduct over the internet is often fragmented across video-conferencing tools, online coding pages and manual notes. This fragmentation makes interviews time-consuming, makes evaluation inconsistent and leaves the interviewer with no reliable way of knowing whether the candidate has remained honest during the session. **Talent Flow: Real-Time Remote Technical Interview & Assessment Platform** is a MERN-stack web application designed to overcome these problems by providing the entire technical interview in a single, browser-based environment.

The platform uses **React.js** with Tailwind CSS for the user interface, **Node.js** and **Express.js** for the back end, and **MongoDB** for data storage. **WebRTC** (through SimplePeer) provides peer-to-peer video and audio, while **Socket.io** provides the real-time channel for signalling, synchronisation of code, delivery of questions and proctoring alerts. The **Monaco Editor** provides a collaborative coding environment with multi-language support, live code execution, custom input and a console. An **AI-based code evaluation** built on the **Groq API** returns a score, complexity remarks, feedback and suggestions for submitted code. **JWT authentication** with role-based access secures the system for its two kinds of users, interviewers and candidates.

The system supports instant and scheduled interviews, a question bank whose questions can be sent to the candidate during the live session, and dashboards for upcoming and past interviews. An **automated proctoring** component detects tab switching and window minimisation, warns the candidate, alerts the interviewer in real time, records each incident and ejects the candidate when the permitted number of alerts is exceeded. At the end of the interview the interviewer records ratings, comments and a decision, and the system generates a detailed **PDF scorecard** with jsPDF and html2canvas that combines scores, AI reviews, proctoring incidents and feedback.

The project was developed through a systematic methodology of requirements analysis, design (data flow, ER and UML diagrams), front-end and back-end development, database and AI integration, testing and deployment. The result is an integrated, low-cost platform that reduces the effort of organising interviews, promotes consistent evaluation and improves the integrity and documentation of remote technical hiring.

Keywords: Remote technical interview, MERN stack, WebRTC, Socket.io, collaborative code editor, Monaco Editor, Groq API, AI code evaluation, automated proctoring, JWT authentication, MongoDB, PDF report generation.

Mapping with Sustainable Development Goals (Mention the Goal)	SDG 8 Decent Work and Economic Growth SDG 9 Industry, Innovation and Infrastructure SDG 10 Reduced Inequalities
--	---

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	List of Figures	v
	List of Tables	vi
	List of Abbreviations	vii
Chapter 1	INTRODUCTION	1
1.1	Background	1
1.2	Motivation	1
1.3	Scope & Objectives	2
1.4	Problem Statement	3
Chapter 2	REQUIREMENT AND ANALYSIS	4
2.1	Existing System & Problem Statement	4
2.2	Solution Overview	5
2.3	Feasibility Study	6
Chapter 3	LITERATURE REVIEW	8
3.1	Intelligent Candidate Monitoring and Interview Surveillance System (2026)	8
3.2	Secure Real-Time Coding Interview System with AI-Based Evaluation and Monitoring (2026)	8
3.3	An Intelligent Online Interview System Based on AI	9
3.4	AI-Based Automatic Interview Assessment	9
3.5	Online Remote Proctoring System Using AI	10
3.6	Collaborative Online Code Editor Using WebRTC and Socket.IO	10
3.7	Remote Technical Interview Platform with Real-Time Collaboration	10
3.8	Other Related Works	10
3.9	Summary of the Literature Survey	11
3.10	Research Gap Identified	12
Chapter 4	DIAGRAMS	13
4.1	Data Flow Diagrams (DFD)	13
4.1.1	DFD LEVEL 0	13
4.1.2	DFD LEVEL 1	13
4.2	Use Case Diagram	17
4.3	Class Diagram	18
4.4	State Diagram	19

CHAPTER NO.	TITLE	PAGE NO.
4.5	Activity Diagram	20
4.6	Sequence Diagram	21
Chapter 5	METHODOLOGY	22
5.1	Module Descriptions	22
5.2	Algorithms	27
5.3	Architecture	29
5.4	Case Study	32
Chapter 6	IMPLEMENTATION DETAILS	34
6.1	Implementation Procedures	34
6.2	Testing Methods and Results	35
Chapter 7	ADVANTAGES AND DISADVANTAGES	37
7.1	Advantages	37
7.2	Disadvantages	38
Chapter 8	RESULT AND DISCUSSION	39
8.1	Analysis	39
Chapter 9	CONCLUSION	41
9.1	Summary	41
9.2	Limitations	41
9.3	Future Scope	42
9.4	Applications	43
Chapter 10	APPENDICES	44
10.1	Sample code	44
Chapter 11	REFERENCES	46

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1.1	Level 0 DFD of Talent Flow	13
4.1.2	Level 1 DFD: Interviewer	14
4.1.3	Level 1 DFD: Interviewer evaluation and report management	15
4.1.4	Level 1 DFD: Candidate	15
4.1.5	Level 1 DFD: AI code evaluation	16
4.1.6	Level 1 DFD: Proctoring and integrity monitoring	16
4.2	Use case diagram of Talent Flow	17
4.3	Class diagram of Talent Flow	18
4.4	State diagram of an interview	19
4.5	Activity diagram of Talent Flow	20
4.6	Sequence diagram of Talent Flow	21
5.1.1	Module structure of Talent Flow	22
5.1.2	Registration (Create Account, left) and login (Welcome Back, right) interfaces	23
5.1.3	Candidate dashboard	24
5.1.4	Interviewer dashboard	25
5.1.5	Live interview room with video, questions and alerts panel, and code editor	26
5.3.1	System architecture of Talent Flow	30
5.3.2	Layered technology architecture of Talent Flow	32

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	- Artificial Intelligence
API	- Application Programming Interface
CORS	- Cross-Origin Resource Sharing
CPU	- Central Processing Unit
CRDT	- Conflict-free Replicated Data Type
CSS	- Cascading Style Sheets
DB	- Database
DFD	- Data Flow Diagram
DOM	- Document Object Model
ER	- Entity-Relationship
HR	- Human Resources
HTML	- HyperText Markup Language
HTTP(S)	- HyperText Transfer Protocol (Secure)
ICE	- Interactive Connectivity Establishment
IDE	- Integrated Development Environment
JSON	- JavaScript Object Notation
JWT	- JSON Web Token
LLM	- Large Language Model
MCA	- Master of Computer Applications
MERN	- MongoDB, Express.js, React.js, Node.js
NAT	- Network Address Translation
NoSQL	- Not Only SQL
ODM	- Object Document Mapping
P2P	- Peer-to-Peer
PDF	- Portable Document Format
PO	- Program Outcome
PSO	- Program Specific Outcome
REST	- Representational State Transfer
RTC	- Real-Time Communication
SDG	- Sustainable Development Goal
SDP	- Session Description Protocol
SPA	- Single Page Application
SRTP	- Secure Real-time Transport Protocol
STUN	- Session Traversal Utilities for NAT
TURN	- Traversal Using Relays around NAT
UI	- User Interface
UML	- Unified Modeling Language
WebRTC	- Web Real-Time Communication
WSS	- WebSocket Secure

CHAPTER 1

INTRODUCTION

1.1 Background

Recruitment in the software industry has moved steadily from face-to-face meetings towards remote, technology-mediated hiring. Organisations now receive applications from candidates located in different cities and countries, and it is common for the entire selection process, from screening to the final technical round, to be conducted online. Among the stages of selection, the technical interview is the most demanding. It must reveal not only what a candidate knows, but how the candidate reasons about a problem, structures a solution, writes and debugs code under time pressure, and communicates while doing so.

A meaningful technical interview therefore requires several capabilities working together: live audio and video so that the interviewer and candidate can talk naturally; a shared coding environment in which both parties see the same code as it is typed; a way to run that code and observe its output; a bank of questions from which the interviewer can choose; and a structured method of recording feedback and scores. In practice these capabilities are usually spread across different products. A video-conferencing tool carries the conversation, an online judge or shared document holds the code, spreadsheets or e-mails hold the feedback, and the interviewer relies on personal judgement to decide whether the candidate has behaved honestly.

Recent advances in web technology have made it possible to bring these capabilities into a single browser-based application. Web Real-Time Communication (WebRTC) allows browsers to exchange audio, video and data directly without plug-ins. WebSocket-based libraries such as Socket.io provide low-latency, bidirectional messaging between the browser and the server. Browser-hosted editors such as the Monaco Editor, which powers Visual Studio Code, offer a professional code-editing experience inside a web page. At the same time, large language models made available through fast inference services such as the Groq API make it feasible to obtain an automatic, explainable review of source code within seconds. These developments form the technical foundation of the present project.

1.1 Motivation

The motivation for this project comes from the everyday difficulties faced by both sides of a remote interview. Candidates often lose valuable minutes of a short interview in switching between links, sharing screens or fixing audio problems, and their performance is judged in an environment that is unfamiliar and fragmented. Interviewers, on the other hand, have

to concentrate simultaneously on the conversation, the candidate's code, the time and their own notes, and they seldom have a systematic way of comparing candidates against a common standard.

Existing commercial solutions address parts of this problem. Video-conferencing tools deliver excellent communication but no coding environment; online coding platforms deliver excellent code execution but often require paid enterprise options for integrated video and offer little help in grading solutions while the interview is in progress. Many of these products are also expensive for small organisations, training institutes and colleges. There is thus a clear opportunity for an open-technology platform that unifies these services, uses artificial intelligence to reduce the interviewer's workload and bias, and monitors interview integrity in an unobtrusive manner.

The project also offered the developer a valuable opportunity to apply a broad set of modern web technologies (component-based user interfaces, REST and WebSocket APIs, peer-to-peer media, NoSQL data modelling and large-language-model integration) to a realistic problem with immediate practical relevance.

1.2 Scope & Objectives

The main objective of Talent Flow is to develop a single web-based platform for conducting remote technical interviews in a secure, organised and efficient manner. The platform brings important interview activities such as audio and video communication, collaborative coding, code execution, AI-based code evaluation, proctoring, interview scheduling and report generation together in one system. This reduces the need for interviewers and candidates to switch between different applications during an interview.

The scope of Talent Flow covers the major stages of a remote technical interview, beginning with user authentication and interview scheduling and continuing through the live interview, coding assessment, monitoring and final evaluation. The platform provides real-time audio, video and screen-sharing facilities using WebRTC. It also includes a collaborative coding environment based on Monaco Editor, where code changes are synchronised between participants using Socket.io. Candidates can write programs, provide input, execute code in supported programming languages and view the results through the console.

Talent Flow also includes an AI-assisted code review feature using the Groq API. The submitted code can be analysed to provide an evaluation, score, complexity-related observations, feedback and suggestions for improvement. The system also provides basic

automated proctoring by monitoring activities such as tab switching and window minimisation. When such activities are detected, the candidate receives a warning and the incident is recorded. If the number of violations exceeds the configured limit, the interview session can be terminated.

The platform provides a question bank through which interviewers can create, manage and send technical questions to candidates during the interview. It also supports both instant interviews using meeting codes and scheduled interviews with invitations. After an interview is completed, the system brings together interviewer ratings, comments, AI-based code evaluation and recorded proctoring incidents to prepare a downloadable PDF scorecard.

Security is maintained through JWT-based authentication and role-based access control, ensuring that users can access only the features permitted for their role. The system is mainly intended for companies, educational institutions and training organisations that conduct technical interviews or coding assessments remotely. Advanced features such as face and gaze tracking, complete video recording of interview sessions and integration with external applicant-tracking systems are not included in the current version and can be considered for future development.

1.3 Problem Statement

Traditional technical interviews rely on several separate tools and on manual processes, which makes interview management difficult and time-consuming. Interviewers are forced to switch between a video-conferencing application, a shared coding page and a note-taking tool, and candidates must learn and configure each of them. Manual evaluation of code introduces delays and inconsistency, because two interviewers reviewing the same solution may award different scores and because detailed review is not possible during the limited time of a live session. Ensuring the integrity of a remote interview is also challenging, since the interviewer cannot see what else the candidate is doing on the computer, and switching to another tab or window to search for answers is easy and hard to detect.

The absence of a centralised platform additionally makes it hard to schedule interviews, keep a reusable set of questions, retain a record of what happened in each session, and produce a consistent report for the hiring team. The problem addressed by this project is therefore the lack of an integrated, real-time and automated platform that supports the whole life cycle of a remote technical interview, from scheduling to the final scorecard, while providing intelligent assistance in evaluation and monitoring of candidate integrity.

CHAPTER 2

REQUIREMENT AND ANALYSIS

2.1 Existing System and Problem Statement

Remote technical interviews are commonly conducted using a combination of different applications for communication, coding, evaluation and documentation. In a typical process, an interviewer uses a video-conferencing platform to communicate with the candidate and shares a programming problem through the call or a separate document. The candidate then writes the solution using an online coding platform or a local development environment and may share the screen so that the interviewer can observe the coding process. After completing the task, the code is reviewed manually, while interview notes, ratings and feedback are often maintained separately. Although these individual tools provide useful functions, they do not offer a unified environment for managing the complete interview process.

Using multiple applications can make the interview process less convenient and more time-consuming. The interviewer and candidate may have to move between different windows for video communication, coding, testing programs and discussing questions. Screen sharing provides visibility into the candidate's work, but it does not provide a proper collaborative coding environment where both participants can interact with the same code in real time. Similarly, general-purpose video-conferencing applications do not normally include features such as technical question management, integrated code execution, AI-assisted code evaluation or interview-specific reporting.

Another challenge is the manual nature of candidate evaluation. Interviewers generally have to observe the candidate's performance, review the submitted code and prepare their own notes and assessment. The final feedback may be stored in documents, spreadsheets or other applications, making it difficult to maintain all interview-related information in one location. Preparing a formal report after every interview can also require additional time and effort. The absence of integrated evaluation support may make the assessment process less consistent when a large number of candidates are interviewed.

Remote technical assessments also require suitable mechanisms for maintaining the integrity of the interview. In a normal video-conferencing session, the interviewer may not

have a dedicated way to identify activities such as switching browser tabs or minimising the interview window. Even when such activities are noticed, recording and reviewing the incidents manually can be difficult. This creates a need for a system that can monitor selected browser activities and provide appropriate warnings and records during the interview.

These limitations indicate the need for a centralised platform that can bring communication, coding, assessment, monitoring and documentation together. Talent Flow is proposed to address this problem through a single web-based environment that supports real-time audio and video communication, collaborative coding, code execution, AI-assisted code review, question management, interview scheduling and automated proctoring. The system also combines interviewer feedback, code evaluation and recorded incidents into a downloadable PDF scorecard. By integrating these activities into one platform, Talent Flow aims to provide a more organised and efficient approach to conducting and managing remote technical interviews.

2.2 Solution Overview

Talent Flow is proposed as a unified web-based platform designed to simplify and improve the process of conducting remote technical interviews and assessments. Instead of depending on separate applications for video communication, coding, evaluation and interview documentation, the system brings these activities together within a single environment. The platform provides dedicated access for Interviewers and Candidates, allowing both participants to manage and conduct technical interviews through an organised workflow.

The system supports real-time audio and video communication so that the Interviewer and Candidate can interact during the interview without requiring an external meeting application. Along with communication, Talent Flow provides a collaborative coding environment where the candidate can write and modify programs while the interviewer can observe the coding process. The integrated code execution facility allows programs to be tested with suitable inputs and the resulting output or errors to be viewed within the same interface. This reduces the need to switch between multiple tools during a technical assessment.

Talent Flow also provides features for managing technical questions and conducting structured assessments. Interviewers can maintain questions based on different topics and difficulty levels and use them during an interview session. The platform further supports AI-assisted code evaluation, which can analyse submitted solutions and provide information such as code quality, possible improvements and complexity-related observations. This information can assist the interviewer during the assessment while keeping the final evaluation under the interviewer's control.

To improve the reliability of remote assessments, the platform includes automated proctoring features. Selected activities such as switching away from the interview tab or minimising the interview window can be detected and recorded as incidents. Appropriate warnings can be shown to the candidate, while the interviewer can review the recorded incidents during or after the session. This provides an additional layer of monitoring for maintaining the expected conduct of a remote technical interview.

The platform also supports interview scheduling, candidate management, feedback and report generation. After an interview, the interviewer can record ratings, comments, strengths and areas for improvement. Relevant interview information, including assessment details, coding results, AI-based observations and proctoring incidents, can be organised into a final scorecard that can be generated as a PDF. By combining these functions into one platform, Talent Flow provides a more structured, convenient and efficient approach to managing the complete remote technical interview process.

2.3 Feasibility Study

The feasibility study is conducted to determine whether the proposed Talent Flow system can be developed and used successfully with the available technologies, resources and time. It examines the technical requirements, expected cost and practical usability of the system. Since Talent Flow combines interview communication, collaborative coding, code execution, evaluation, proctoring and report generation, checking its feasibility helps ensure that the proposed solution can be implemented effectively.

2.3.1 Technical Feasibility

Talent Flow is technically feasible because its major functions can be developed using commonly available web development technologies and tools. The system can support real-

time communication, collaborative coding, database management, code execution and AI-assisted evaluation through suitable web technologies, frameworks and APIs. Development and testing can be carried out using standard computers with an internet connection, webcam and microphone. The required software tools and development frameworks are also readily available, making the technical requirements manageable for the project.

2.3.2 Economic Feasibility

The proposed system is economically feasible because it can be developed without requiring expensive specialised hardware. Most of the development work can be carried out using open-source frameworks, libraries and commonly available development tools. The system also reduces the need to depend on multiple separate applications for conducting interviews, coding and assessment. Although some external services or APIs may involve usage costs during large-scale deployment, the basic development and testing of Talent Flow can be completed with limited financial resources.

2.3.3 Operational Feasibility

through the platform, while Candidates can join interviews, communicate with the interviewer and complete coding tasks in the same environment. The centralised design reduces unnecessary switching between applications and makes interview information easier to manage. Since the system is web-based, users can access it through a supported browser without installing a separate desktop application. Therefore, the proposed system can be practically used for conducting and managing remote technical interviews.

CHAPTER 3

LITERATURE REVIEW

A literature survey was carried out to understand how earlier researchers and developers have approached the individual capabilities that Talent Flow combines: intelligent monitoring of remote candidates, collaborative code editing, AI-assisted evaluation of interviews and proctoring of online examinations. The surveyed works are summarised in the following sections, and the relevance of each to the present project is discussed. The descriptions are based on the titles, techniques and findings reported for each work in the literature survey conducted for this project.

3.1 Intelligent Candidate Monitoring and Interview Surveillance System (2026)

This work, authored by Dr. T. Manoj Prasath and colleagues and published in the *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, focuses on maintaining the integrity of remote interviews through automatic surveillance of the candidate. The system combines the YOLO object-detection model, Google's MediaPipe framework and machine-learning classifiers.

- **Face tracking and gaze estimation:** the candidate's face and eye direction are followed to determine whether attention remains on the screen.
- **Object detection:** YOLO is used to identify unauthorised objects, such as a mobile phone, appearing in the camera view.
- **Alerts:** when suspicious behaviour is found, alerts are generated for the interviewer.
- **Evaluation metric:** the system is assessed using detection accuracy, and the reported outcome is improved integrity of remote interviews.

Relevance: the study establishes that automated monitoring measurably improves interview integrity. Talent Flow adopts the same principle of real-time alerts, but uses lightweight browser-event monitoring (tab switching and window minimisation) so that no additional model has to be run on the candidate's machine. Camera-based analysis of the kind described here is identified as a future enhancement.

3.2 Secure Real-Time Coding Interview System with AI-Based Evaluation and Monitoring (2026)

Published by Harshavardhan and co-authors in the *International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE)*, this system is the closest of the surveyed works to the present project. It combines Conflict-free Replicated Data Types (CRDT), WebSockets, artificial intelligence and Docker to build a secure coding-interview platform.

- **Collaborative editor:** CRDT-based synchronisation allows the interviewer and the candidate to edit the same code without conflicts.
- **AI evaluation:** submitted code is assessed by an AI component to support the interviewer's judgement.
- **WebRTC and secure execution:** live video is provided through WebRTC, and code is executed in a controlled, containerised environment.
- **Behavioural monitoring:** candidate behaviour is monitored during the session.
- **Metrics and result:** collaboration accuracy and quality of AI evaluation are measured, and the authors report an integrated interview platform with secure coding and monitoring.

Relevance: this work validates the architectural idea of integrating real-time collaboration, AI evaluation and monitoring into one platform. Talent Flow follows the same integrated approach on the MERN stack, using Socket.io for synchronisation and the Groq API for evaluation. The use of containers for secure code execution and CRDT for conflict-free editing are noted as directions for further hardening of the system.

3.3 An Intelligent Online Interview System Based on AI

This IEEE publication proposes an online interview system in which machine-learning techniques assist in automating the interview process and in assessing candidates. The features highlighted are AI-assisted interview automation and candidate assessment, and the performance measure used is assessment accuracy. The reported outcome is improved consistency of interviews.

Relevance: consistency is one of the central concerns of Talent Flow. By presenting every candidate with questions from a common question bank and by generating an AI review with the same structure (score, complexity, feedback and suggestions) for every submission, the proposed platform aims to achieve similar consistency in technical assessment.

3.4 AI-Based Automatic Interview Assessment

This IEEE work applies natural language processing and deep learning to the automatic evaluation of candidates' responses. Precision and recall are used as performance measures, and the reported benefit is a reduction in evaluator bias, because the same automated criteria are applied to every candidate.

Relevance: the study supports the use of automated evaluation to complement human judgement. In Talent Flow the automated evaluation is applied to programming answers using a large language model, and the result is presented to the interviewer as a recommendation. The final decision remains with the interviewer.

3.5 Online Remote Proctoring System Using AI

This IEEE paper describes a proctoring system based on computer vision that monitors examinees remotely and detects malpractice. The measure of success is detection rate, and the finding is enhanced security of the assessment.

Relevance: it demonstrates the value of continuous, automated supervision when no human invigilator is physically present. Talent Flow applies the same idea to interviews through detection of tab switching and window minimisation, together with a warning mechanism and an incident log.

3.6 Collaborative Online Code Editor Using WebRTC and Socket.IO

This IEEE publication builds an online code editor that permits several users to program together in real time, using WebRTC and Socket.IO as the underlying technologies. The performance metric is synchronisation latency, and the reported result is improved collaborative coding.

Relevance: the technology combination used in this work is the same as that chosen for Talent Flow. It confirms that Socket.io is suitable for keeping editor content synchronised with low delay and that WebRTC is suitable for the accompanying audio and video.

3.7 Remote Technical Interview Platform with Real-Time Collaboration

This IEEE work presents a remote technical-interview platform built on WebSockets. It offers live coding, chat and an interviewer dashboard, and evaluates the system through response time. The result reported is a better interview experience for both participants.

Relevance: the feature set of live coding, chat and a dashboard for the interviewer corresponds closely to the core of Talent Flow. The present project extends it with AI-based code review, proctoring with warnings and incident tracking, a question bank and PDF report generation.

3.8 Other Related Works

The following additional works were consulted to understand specific design choices.

- **CodeSync (2026), IJCRT:** a real-time collaborative code editor. It reinforces the design of a shared editor whose changes are propagated to all participants of a session.
- **REALCODEAI (2025), IJSRSET:** a platform for real-time code collaboration and interviews. It shows the growing interest in combining live code sharing with AI support for interviews.
- **Code Together (2024):** collaborative code editing using CRDT. CRDTs are data structures that allow replicas of a document to be modified independently and to converge to the same state without central coordination, which makes them attractive for conflict-free editing.

- **Smart AI-Based Online Proctoring System (2022), IEEE:** an intelligent proctoring system for online assessments, supporting the use of automation for detecting suspicious behaviour.
- **AI-Based Fully Automated Online Proctoring Systems: A Survey (2022), IEEE:** a survey of automated proctoring approaches, useful for understanding the range of monitoring techniques and their trade-offs.
- **Deep-Learning-Based Online Exam Proctoring Systems: A Review (2022), IJSRSET:** a review of deep-learning approaches to proctoring, useful for identifying advanced techniques that can be adopted in later versions of Talent Flow.

3.9 Summary of the Literature Survey

Table 3.1: Summary of the surveyed literature

Work (Year)	Source	Technique	Contribution	Outcome
Intelligent Candidate Monitoring and Interview Surveillance System (2026)	IJRASET	YOLO + MediaPipe + ML	Face tracking, object detection, gaze estimation, alerts	Improved remote interview integrity
Secure Real-Time Coding Interview System with AI-Based Evaluation and Monitoring (2026)	IJARCCCE	CRDT + WebSockets + AI + Docker	Collaborative editor, AI evaluation, WebRTC, secure execution, behavioural monitoring	Integrated interview platform with secure coding and monitoring
An Intelligent Online Interview System Based on AI	IEEE	Machine learning	AI-assisted interview automation and candidate assessment	Improved interview consistency
AI-Based Automatic Interview Assessment	IEEE	NLP + deep learning	Automatic evaluation of candidate responses	Reduced evaluator bias
Online Remote Proctoring System Using AI	IEEE	Computer vision	Remote monitoring and malpractice detection	Enhanced assessment security
Collaborative Online Code Editor Using WebRTC and Socket.IO	IEEE	WebRTC + Socket.IO	Real-time collaborative programming	Improved collaborative coding
Remote Technical Interview Platform with Real-Time Collaboration	IEEE	WebSockets	Live coding, chat, interviewer dashboard	Better interview experience

3.10 Research Gap Identified

The survey shows that individual capabilities, namely collaborative editing, AI-supported assessment and remote proctoring, are well studied, and that a few recent works begin to combine them. However, several gaps remain that Talent Flow attempts to fill:

- Most works concentrate on one capability, such as surveillance or collaborative editing, and do not deliver a complete workflow that includes scheduling, a question bank, feedback and reporting.
- Very little of the surveyed work addresses the **post-interview stage**, namely the consolidation of scores, AI reviews, proctoring incidents and feedback into a single document that can be handed to the hiring team.
- Heavy computer-vision monitoring requires additional computation and camera analysis on the candidate's side. A **lightweight, browser-event-based proctoring approach** with a warning limit and automatic termination has received less attention.
- Few solutions are built on a widely available, open technology stack (MERN, WebRTC, Socket.io, Monaco) that colleges and small organisations can deploy and extend at low cost.
- exports it as a PDF.

CHAPTER 4

DIAGRAMS

4.1 Data Flow Diagrams (DFD)

A Data Flow Diagram (DFD) is a graphical representation of how data move through a system. It shows the external entities that supply or receive data, the processes that transform the data, and the data stores in which data are kept. A DFD can be drawn at different levels of detail. The context-level (Level 0) diagram shows the system as a single process, and the Level 1 diagrams break that process into its main sub-processes. The DFDs of Talent Flow use request and response arrows between the entities, processes and data stores.

4.1.1 DFD Level 0 (Context Diagram)

Figure 4.1.1 shows the context diagram. The whole platform is represented by one process, *Talent Flow*, with two external entities: the **Interviewer** and the **Candidate**. Each entity sends requests to the platform and receives responses. The interviewer requests actions such as creating and monitoring interviews and generating reports; the candidate requests actions such as joining an interview and submitting code.

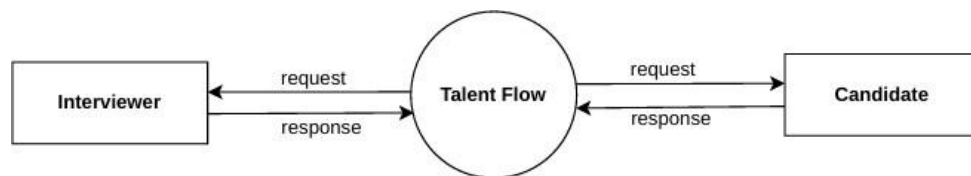


Figure 4.1.1: Level 0 DFD of Talent Flow

4.1.2 DFD Level 1: Interviewer

Figure 4.1.2 expands the interviewer's interaction after login into six processes: **1.0 Manage Candidate**, **2.0 Manage Interview**, **3.0 Schedule Interview**, **4.0 Manage Questions**, **5.0 Monitor Interview** and **6.0 Evaluate Candidate**. These processes read from and write to the data stores for candidates, interviews and questions.

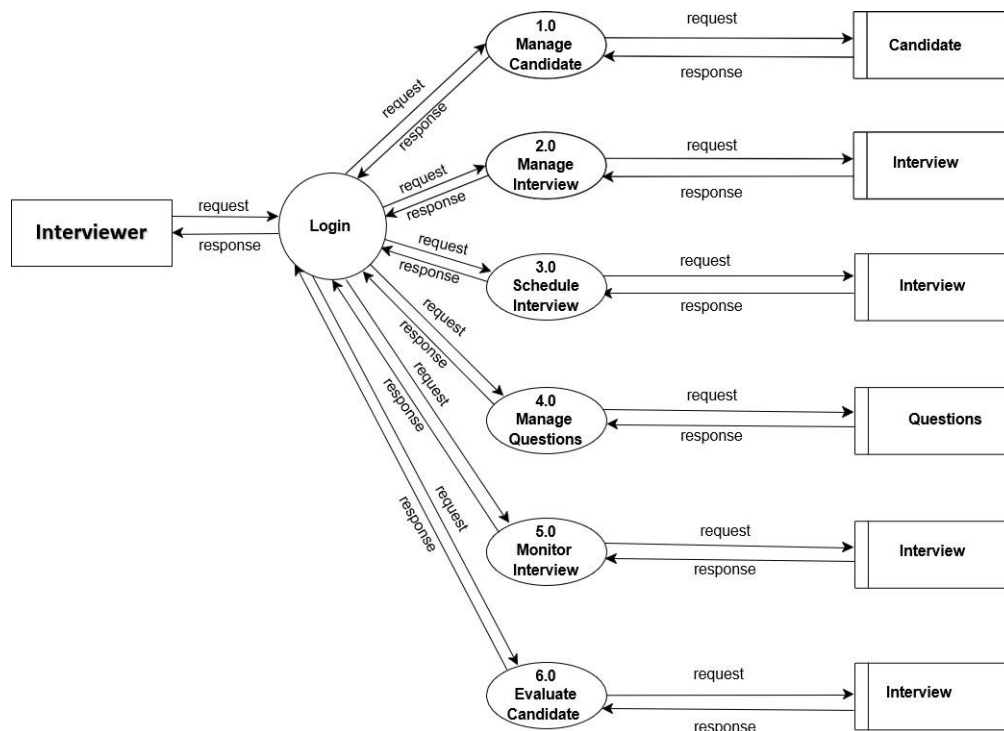


Figure 4.1.2: Level 1 DFD: Interviewer

4.1.3 Level 1 DFD: Interviewer Evaluation and Reporting

Figure 4.1.3 shows the report-management part of the interviewer's activity. The processes are **1.0 Aggregate Data** from the interview, **2.0 Evaluate Performance** using AI evaluation data, **3.0 Proctoring Analysis** using the proctoring logs, **4.0 Feedback and Score**, **5.0 Generate PDF Report** and **6.0 Download Report**.

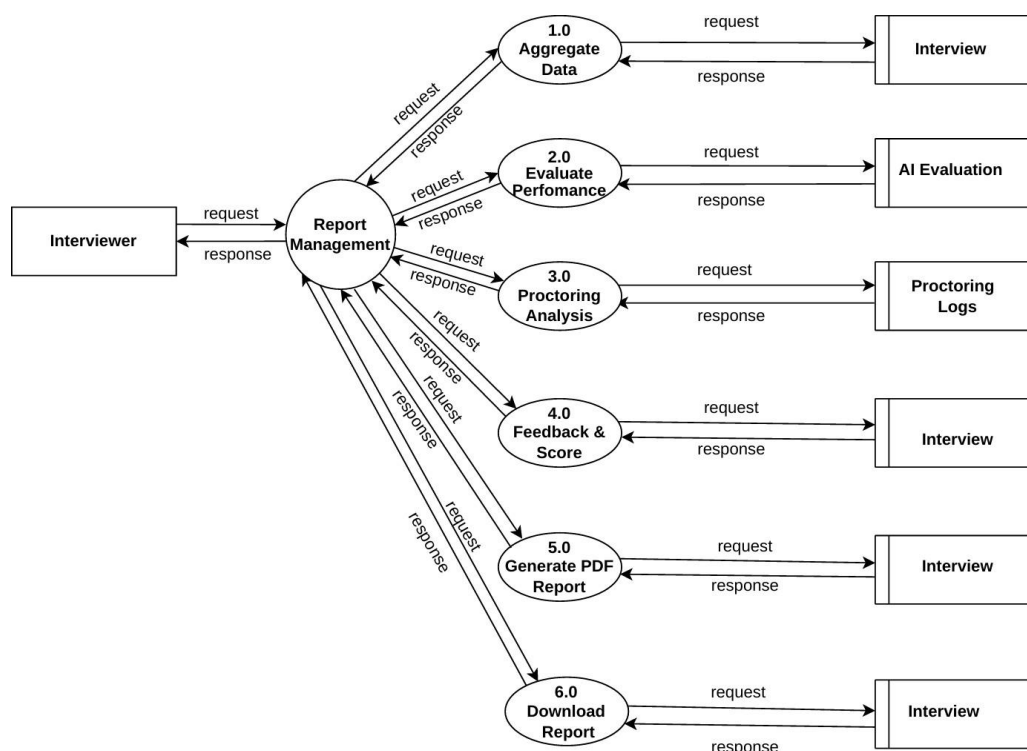


Figure 4.1.3: Level 1 DFD: Interviewer evaluation and report management

4.1.4 Level 1 DFD: Candidate

Figure 4.1.4 shows the candidate's interaction after login. The processes are **1.0 Join Interview**, **2.0 Code Editor**, **3.0 Real Time Communication** and **4.0 Submit Code**. Code entered in the editor flows to the AI evaluation store, and submitted code is recorded against the interview.

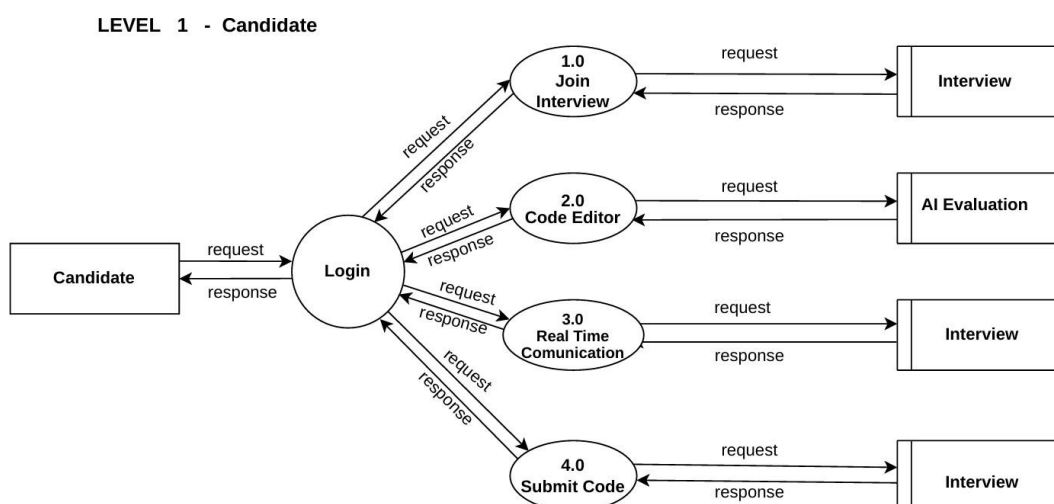


Figure 4.1.4: Level 1 DFD: Candidate

4.1.5 Level 1 DFD: AI Code Evaluation

Figure 4.1.5 represents the AI evaluation subsystem. The processes are **1.0 Receive Code**, **2.0 Code Evaluate**, **3.0 Answer Analysis**, **4.0 Generate AI Review** and **5.0 Send Review**

Result. The AI system exchanges data with the interview and AI-evaluation stores and returns the review to the interview.

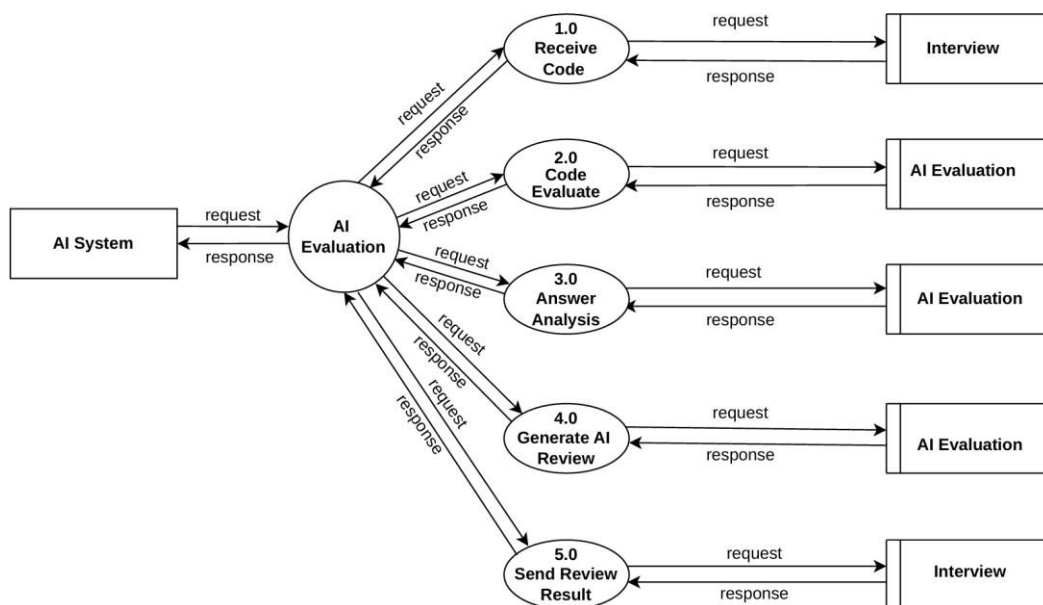


Figure 4.1.5: Level 1 DFD: AI code evaluation

4.1.6 Level 1 DFD: Proctoring and Integrity Monitoring

Figure 4.1.6 shows the proctoring subsystem. The design comprises **1.0 Face Detection**, **2.0 Tab Switch Detection**, **3.0 Copy/Paste Monitoring**, **4.0 Suspicious Activity Analysis** and **5.0 Warning and Ejection Trigger**. In the present implementation the active detections are tab switching and window minimisation, and the warning and ejection logic; face detection and copy/paste monitoring are part of the design and are listed in the future scope.

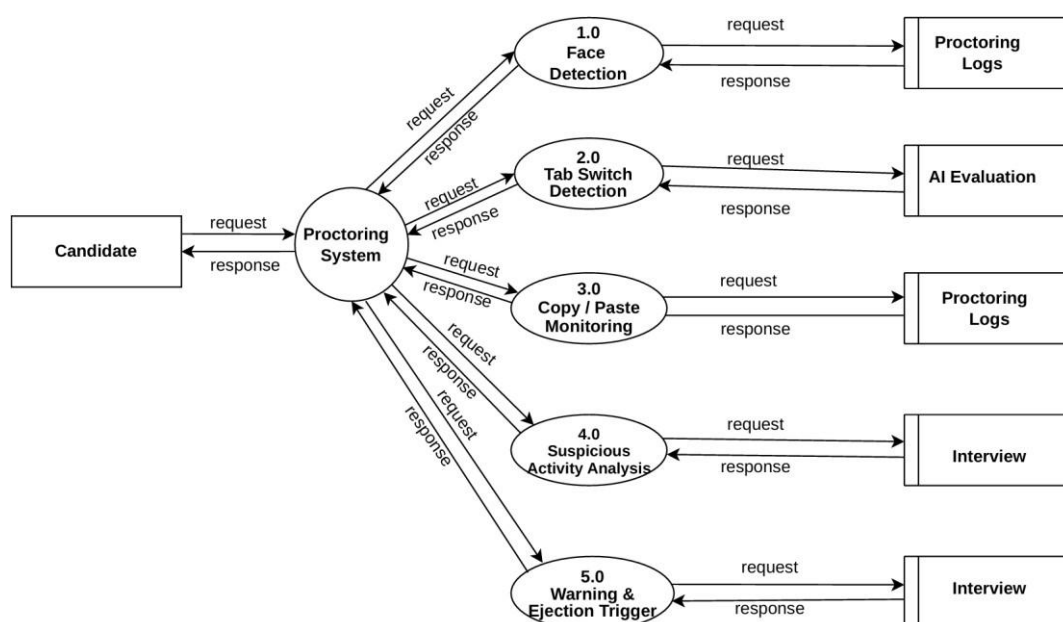


Figure 4.1.6: Level 1 DFD: Proctoring and integrity monitoring

4.2 Use Case Diagram

The use case diagram in Figure 4.2 describes the system from the users' point of view and identifies three actors: the **Interviewer**, the **Candidate** and the **AI System (Groq API)**.

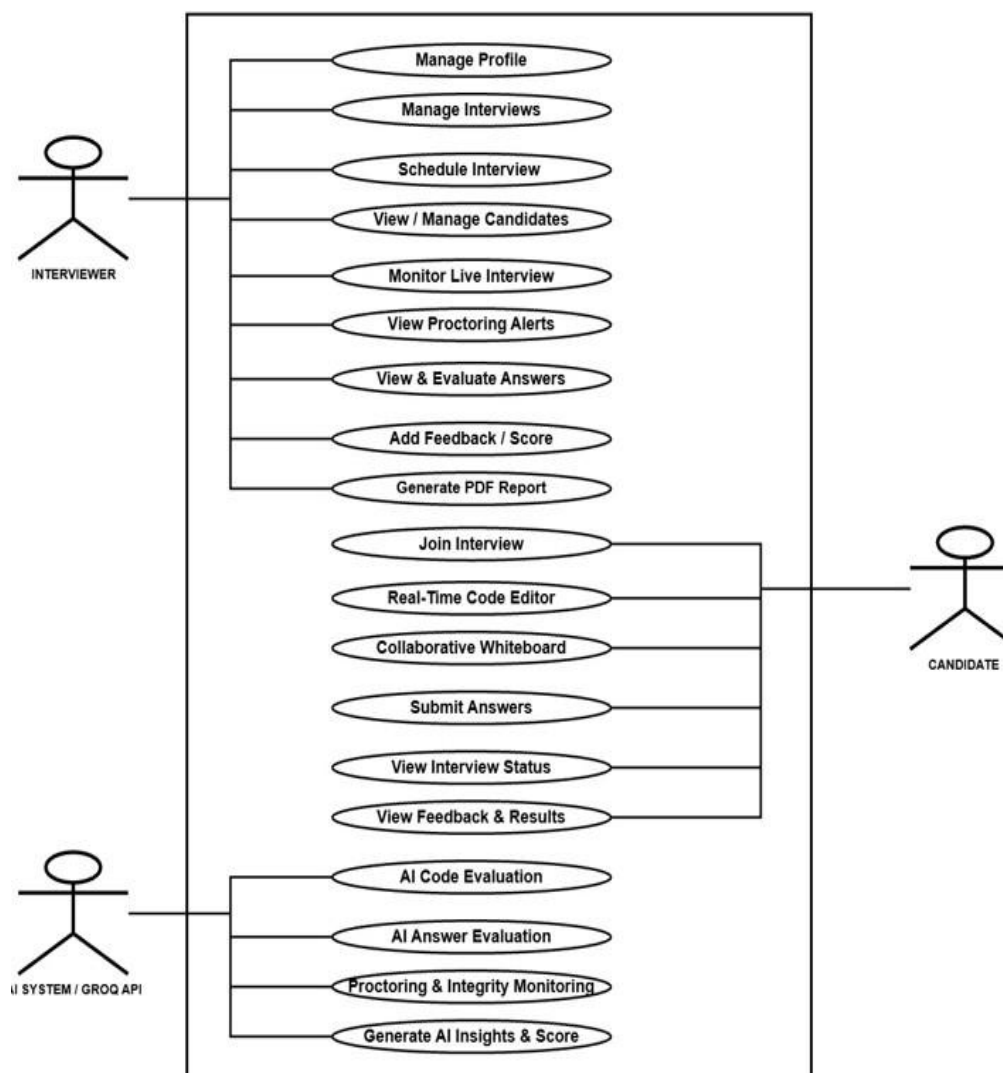


Figure 4.2: Use case diagram of Talent Flow

- **Interviewer:** Manage Profile, Manage Interviews, Schedule Interview, View/Manage Candidates, Monitor Live Interview, View Proctoring Alerts, View and Evaluate Answers, Add Feedback/Score and Generate PDF Report.
- **Candidate:** Join Interview, Real-Time Code Editor, Collaborative Whiteboard, Submit Answers, View Interview Status and View Feedback and Results.
- **AI System:** AI Code Evaluation, AI Answer Evaluation, Proctoring and Integrity Monitoring, and Generate AI Insights and Score.

4.3 Class Diagram

The class diagram in Figure 4.3 depicts the static structure of the system. Each class corresponds to an entity of the ER diagram and lists its attributes and its principal operations. For example, the **Interviewer** class provides login, profile management, interview creation, scheduling, feedback and report generation; the **Candidate** class provides registration, login, joining an interview, submitting code and answers and viewing feedback; the **Interview** class offers create, update status, start and end; the **Interview Session** class offers start, join, end and workspace synchronisation; the **Code Submission** class offers submit, execute and save; the **AI Review** class offers evaluate code, evaluate answer, generate feedback and calculate score; the **Proctoring Log** class offers monitor activity, detect violation, record event and send alert; and the **Report** class offers generate, export PDF and share.

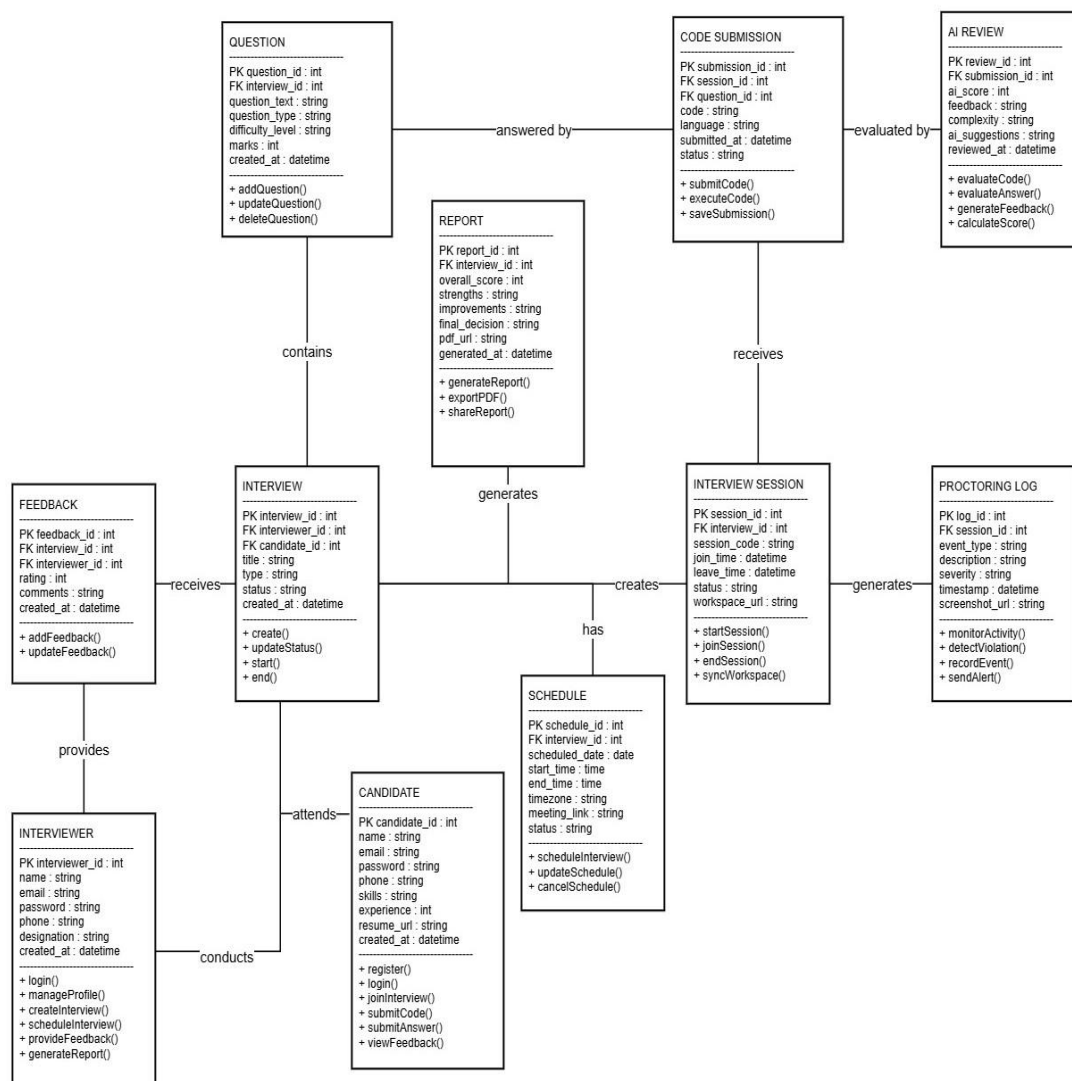


Figure 4.3: Class diagram of Talent Flow

4.4 State Diagram

The state diagram in Figure 4.4 shows the life cycle of an interview. An interview is created, the candidate is invited and the interview becomes *Scheduled*. When the candidate and the interviewer have both joined, the live workspace is initialised and the interview becomes *Active*. In the active state, questions or tasks are presented, the candidate answers and submits code, and the AI evaluation with feedback and score is obtained. In parallel, proctoring monitors the session: if a violation is detected, a warning is issued, and if the violation is serious the candidate is terminated or ejected. When there are no more questions, the interview is completed, the interviewer adds feedback, the final score is calculated, and the interview report and PDF report are generated. An interview may also be cancelled before it starts.

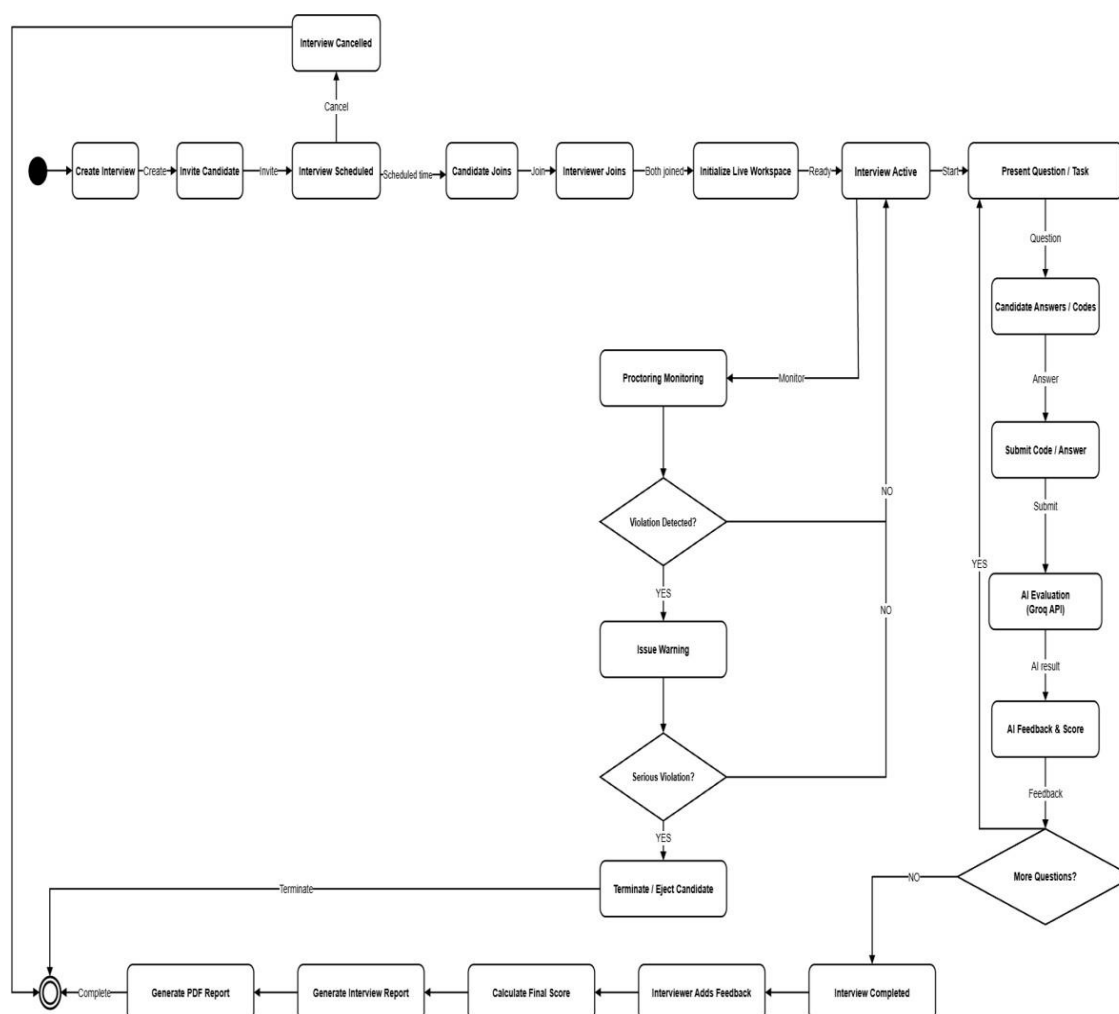


Figure 4.4: State diagram of an interview

4.5 Activity Diagram

The activity diagram in Figure 4.5 uses swim-lanes for the Candidate, the System, the Interviewer and the AI/Proctoring Engine. The candidate registers or logs in, browses interviews, joins the session, participates, submits code or answers and views feedback. The system validates credentials, displays available interviews, confirms the schedule and sends invitations, creates the session and meeting link, receives submissions, stores data and generates logs, aggregates evaluation and scores, updates the interview status and generates the report. The interviewer conducts the interview, reviews submissions, evaluates and scores the candidate, submits the evaluation and feedback and views reports. The proctoring engine detects suspicious behaviour, captures evidence, warns the candidate, alerts the interviewer and, when the warning limit is exceeded, terminates the session and produces an incident report.

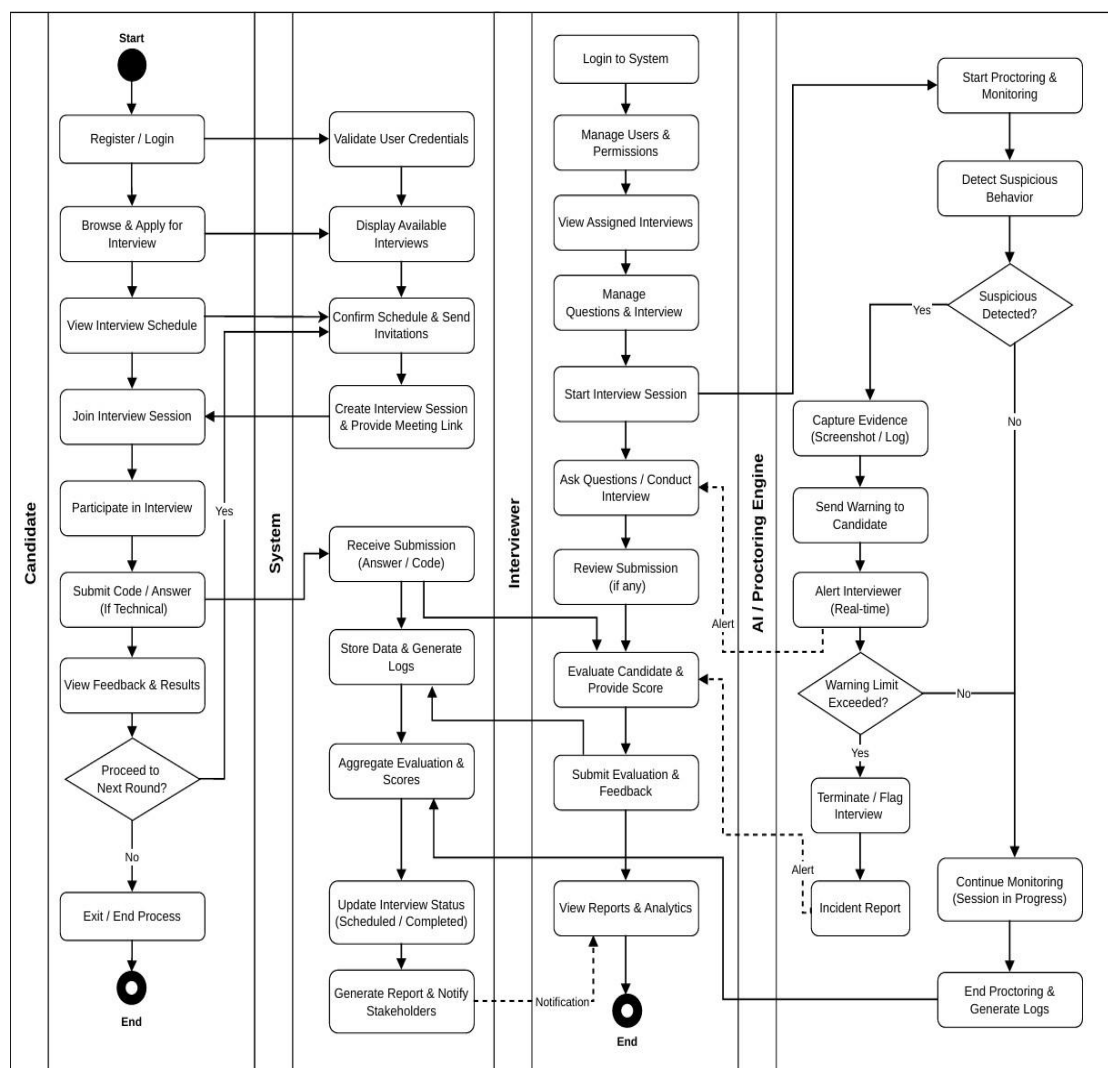


Figure 4.5: Activity diagram of Talent Flow

4.6 Sequence Diagram

The sequence diagram in Figure 4.6 shows the chronological interaction between the Admin (interviewer), the Candidate, the System, the AI and the Database. The interaction is divided into six stages:

1. **Authentication:** the user logs in, the system verifies the credentials with the database and confirms a successful login.
2. **Interview scheduling:** the interviewer schedules an interview, the details are saved and an interview link is sent to the candidate.
3. **Interview session:** the candidate joins the interview and the interviewer starts the session.
4. **AI evaluation:** the candidate submits code or answers, the system requests the AI evaluation, receives the result and saves it.
5. **Proctoring:** the system sends proctoring alerts to the interviewer, and the final evaluation is saved.
6. **Report generation:** the interviewer adds feedback and score, the system retrieves the evaluation data and produces the PDF report.

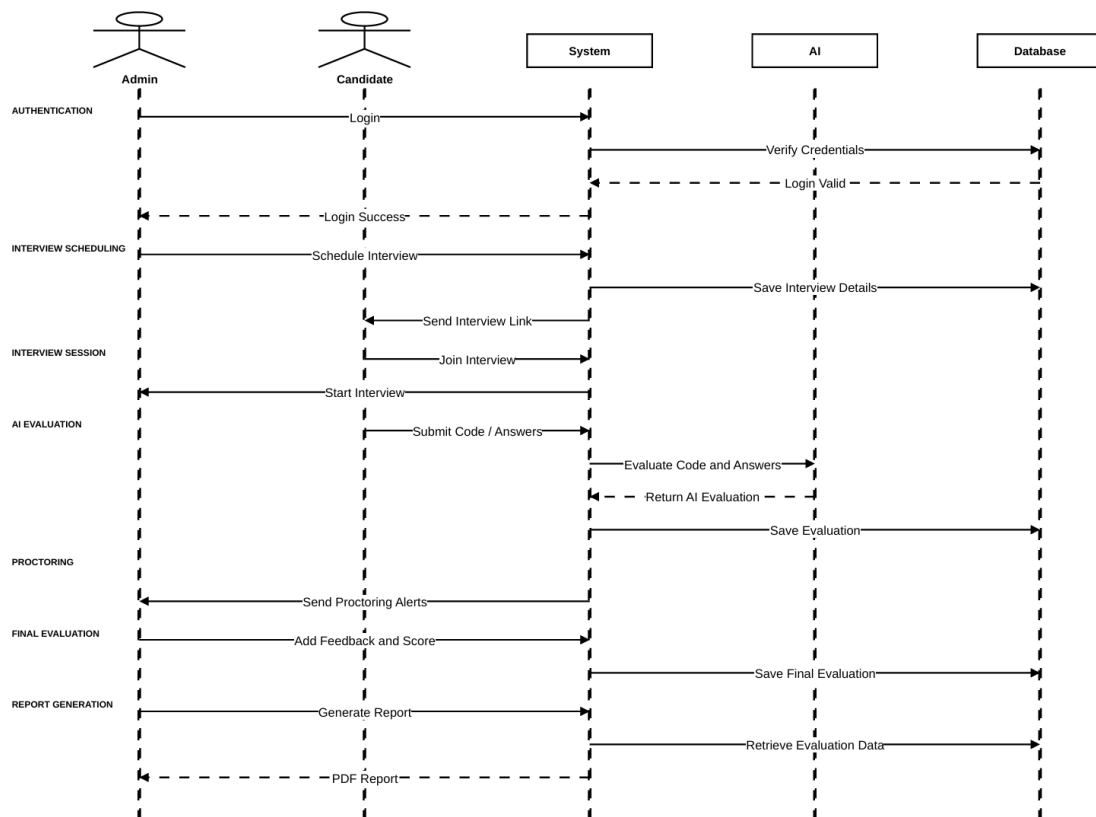


Figure 4.6: Sequence diagram of Talent Flow

CHAPTER 5

METHODOLOGY

5.1 Module Descriptions

The system is divided into six modules, shown in Figure 5.1. Each module groups related functionality, has clear responsibilities and interacts with the others through well-defined APIs and events.

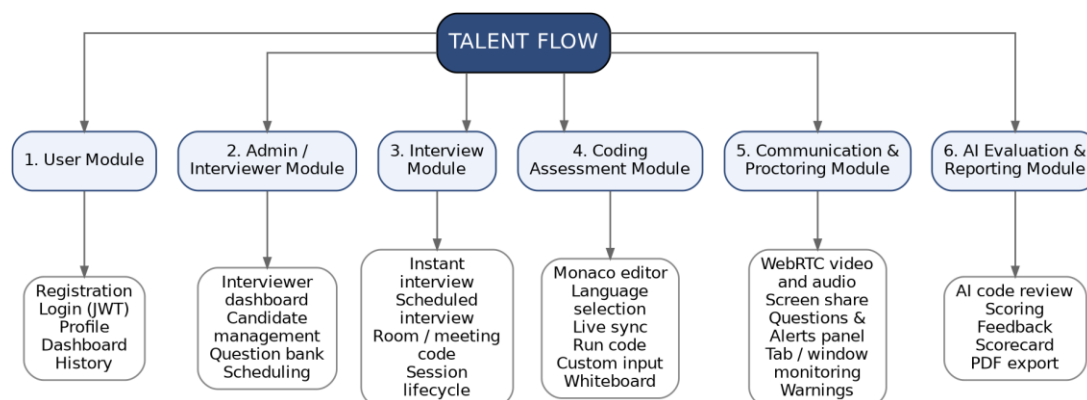


Figure 5.1.1: Module structure of Talent Flow

Table Modules of Talent Flow

No.	Module	Responsibility
1	User Module	Handles all user-related activities: registration, login, profile and dashboards.
2	Admin (Interviewer) Module	Used by the interviewer to manage the platform: candidates, interviews, schedule and question bank.
3	Interview Module	Handles the end-to-end interview process, from room creation to completion.
4	Coding Assessment Module	Manages coding-based evaluation: editor, execution and whiteboard.
5	Communication and Proctoring Module	Handles video, audio, questions and alerts, and monitors the candidate.
6	AI Evaluation and Reporting Module	Provides AI review, scoring, feedback and PDF report generation.

User Module

The User Module manages everything related to the identity of a user. During **registration**, a user provides a full name, an e-mail address and a password; the system checks that the e-mail is not already registered and stores the account with the password in hashed form. **Login** verifies the e-mail and password and returns a signed JWT together with basic user information. The token is stored by the browser and attached to subsequent requests.

After login, the user reaches a role-specific dashboard. Figure 5.1.2 shows the registration and login screens. The registration screen contains the fields Full Name, Email Address and Password with a Sign Up button and a link for existing users. The login screen ("Welcome Back") contains the e-mail and password fields with a Login button and a link to sign up.

The figure displays two side-by-side screenshots of the Talent Flow application interface. Both screens have a dark blue background with a light blue 'Talent Flow' logo at the top.

Left Screen: Candidate Registration

- Header: Talent Flow logo, 'Candidate Registration' title, and subtitle 'Create your profile to join technical assessments'.
- Form Fields:
 - Full Name *: Aromal Anil
 - Email Address *: aromal@gmail.com
 - Password *: [masked]
 - Phone Number: 9400542480
 - Education / Degree: MCA
 - Key Technical Skills: Mongodb, Express, React, Node
- Buttons: A large blue 'Create Account' button at the bottom.
- Link: 'Already have an account? Sign in here' at the bottom.

Right Screen: Welcome Back

- Header: Talent Flow logo, 'Welcome Back' title, and subtitle 'Sign in to access your assessment workspace'.
- Buttons: 'Interviewer Login' and 'Candidate Login' at the top.
- Form Fields:
 - Email Address: aromal@gmail.com
 - Password: [masked]
- Buttons: A large blue 'Sign In' button.
- Link: 'Don't have a candidate account yet? Register as Candidate' at the bottom.

Figure 5.1.2: Registration (Create Account, left) and login (Welcome Back, right) interfaces

The **candidate dashboard**, shown in Figure 5.1.3, welcomes the candidate and offers a "Join Instant Room" field where the meeting code given by the interviewer can be entered. Below it are the "Upcoming" list of scheduled sessions and the "History" list of past sessions with their codes, dates and status.

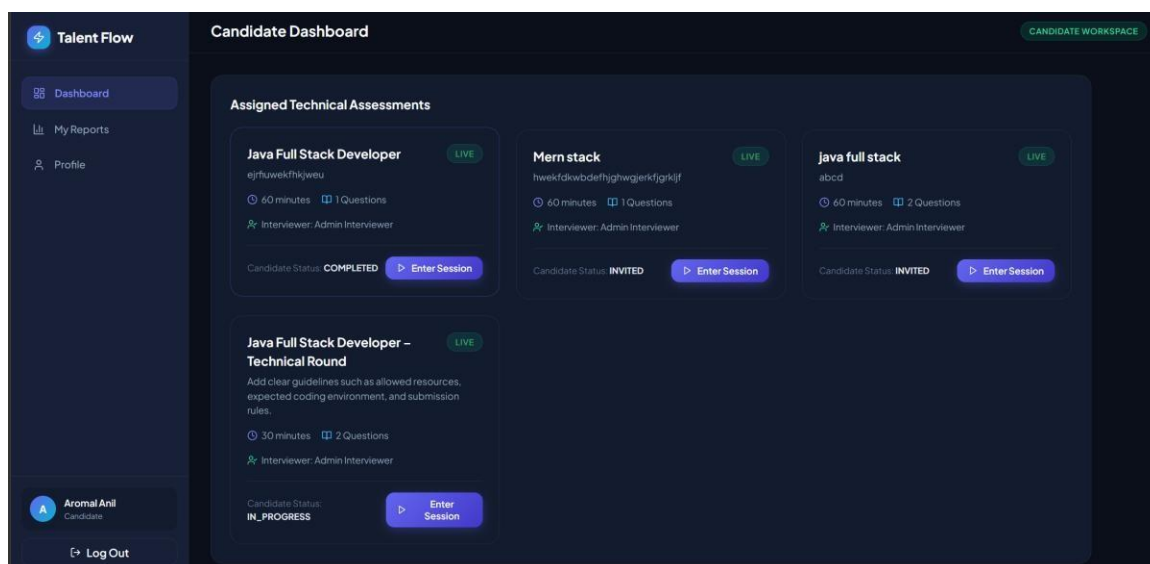


Figure 5.1.3: Candidate dashboard

Admin (Interviewer) Module

The Admin Module is used by the interviewer, who also acts as the administrator of the platform. Its functions are:

- **Profile management:** view and update the interviewer's details, such as name, phone and designation.
- **Interview management:** create interviews, update their status and start or end them.
- **Scheduling:** schedule an interview for a candidate by e-mail with date, time and timezone; cancel or update a schedule.
- **Candidate management:** view candidates and their interview history.
- **Question bank:** add, update and delete questions with text, type, difficulty level and marks, and send questions to the candidate instantly during the live interview.
- **Monitoring and evaluation:** watch proctoring alerts, view answers and AI reviews, add feedback and score, and generate the PDF report.

Figure 5.1.4 shows the Interviewer Dashboard. It presents three action cards, namely **Instant Interview** (Start Meeting), **Schedule Interview** (Schedule) and **Question Bank** (Manage Questions), followed by the lists of upcoming and past interviews with a "View Details" link for each completed interview.

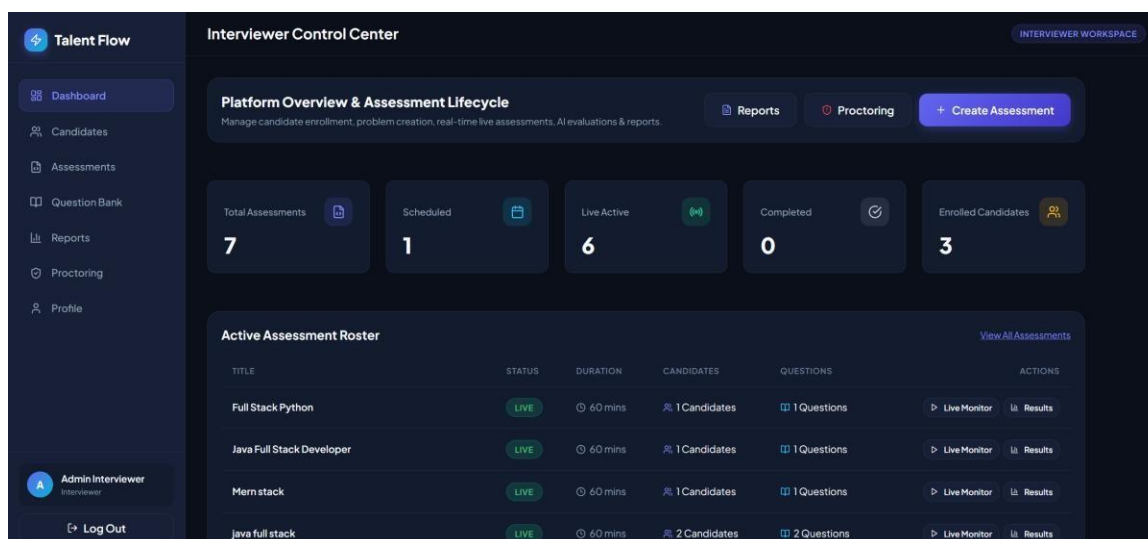


Figure 5.1.4: Interviewer dashboard

Interview Module

The Interview Module controls the life cycle of an interview. The interview is first **created** with a title, a type (technical or HR) and the interviewer and candidate. An **instant interview** immediately produces a session with a unique session code, while a **scheduled interview** carries the schedule and meeting link. When a participant enters the room, the module records the join time; when the interview ends, it records the leave time and updates the status to completed. The states through which an interview passes are shown in the state diagram in Chapter 4.

The module also maintains the **upcoming and history lists** shown on the dashboards. A session is identified by its session code, which the candidate enters to join; the server verifies that the code belongs to an active interview before the candidate is admitted to the room.

Coding Assessment Module

The Coding Assessment Module offers the shared workspace in which the technical part of the interview takes place. It appears in the lower right of the interview room (Figure 5.1.5) as a tabbed area with a **Code Editor** and a **Whiteboard**.

- **Monaco Editor:** provides syntax highlighting, line numbers and editing features for the selected language.
- **Language selection:** a drop-down lists the supported languages (Python is selected by default in the interface).
- **Live synchronisation:** a "Live Sync" indicator shows that changes are being shared with the other participant through Socket.io.

- **Run Code:** sends the code and the custom input to the server and displays the result in the Console Output panel; the Custom Input panel allows values to be supplied to programs that read input.
- **Save:** stores the current code as a submission for later evaluation.
- **Whiteboard:** a drawing area that the participants can use for diagrams and explanations.

Each saved submission is linked to the interview session and to the question that is being answered, and can be evaluated by the AI module.

Communication and Proctoring Module

This module handles both communication and the monitoring of the candidate. The interview room, shown in Figure 5.1.5, is dominated by the video area in which the participants appear. Round controls at the bottom of the video area allow the user to mute the microphone, switch the camera on or off, share the screen and end the call.

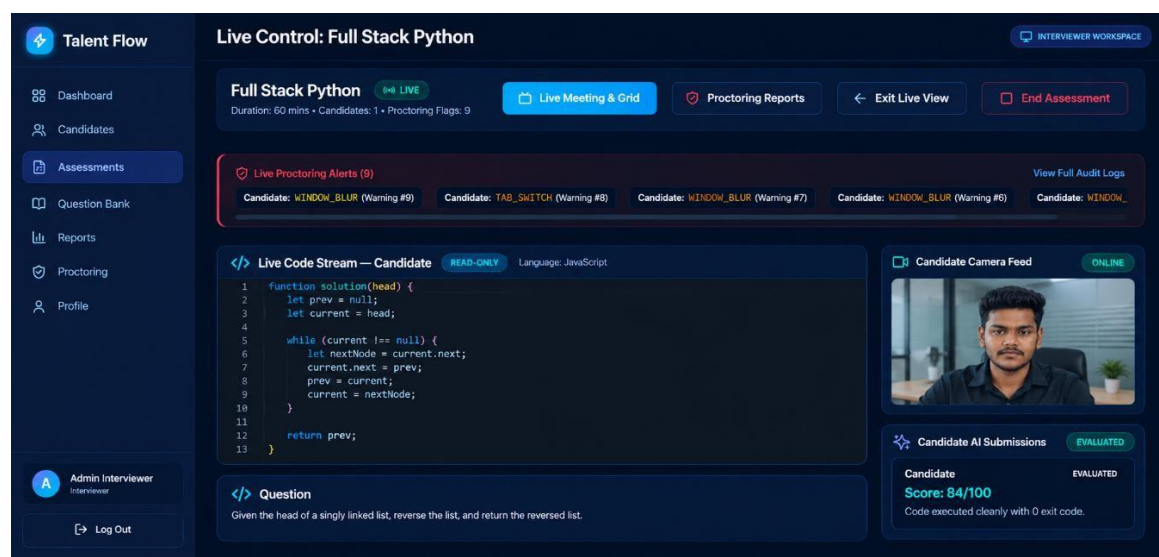


Figure 5.1.5: Live interview room with video, questions and alerts panel, and code editor

On the left side of the room is the **Questions and Alerts** panel, marked "Live". It shows the questions that the interviewer sends from the question bank or types in the input box at the bottom, together with system warnings. On the toolbar of the editor a **Max Alerts** indicator (set to 3 in the figure) shows the number of violations that are permitted.

The proctoring component runs in the candidate's browser. It reacts to tab switching and window minimisation, displays a warning to the candidate, sends an alert to the interviewer in real time, records the incident with its type, severity and time, and increments the alert counter. When the maximum number of alerts is reached, the candidate is terminated or ejected from the session. The flow is described in detail in Chapter 4.

AI Evaluation and Reporting Module

The AI Evaluation and Reporting Module provides intelligent evaluation and the final documentation.

- **AI code review:** when the **AI Review** button is pressed, the question and code are sent to the Groq API. The response is converted into an AI score, complexity information, textual feedback and suggestions and stored as an AI review linked to the submission.
- **Answer evaluation and scoring:** the module can evaluate written answers and calculate a score, supporting the interviewer in arriving at the overall marks.
- **Feedback:** the interviewer adds a rating from 1 to 5 and comments, and can update them later.
- **Report generation:** the module aggregates the interview data, the submissions, the AI reviews, the proctoring log and the feedback into a report with overall score, strengths, improvements and final decision, and exports it as a PDF.

5.2 Algorithms

Talent Flow uses several algorithms and logical procedures to support the major activities of the platform. These algorithms help manage user authentication, interview sessions, coding activities, AI-based code analysis and proctoring. The following algorithms describe the basic processing approach used by the system.

User Authentication Algorithm

The authentication process verifies the login details provided by the user and identifies whether the user is an Interviewer or Candidate.

Steps:

1. Accept the user's login credentials.
2. Validate the entered information.
3. Check the credentials against the stored user details.
4. If the details are valid, generate an authentication token.
5. Identify the user's role.
6. Provide access to the corresponding dashboard.
7. If the credentials are invalid, display an appropriate error message.

Interview Session Algorithm

This algorithm manages the creation and joining of an interview session.

Steps:

1. The Interviewer creates or schedules an interview.
2. Generate a unique interview or session identifier.
3. Store the interview details.
4. Allow the Candidate to join using the provided information.
5. Verify the session and participant details.
6. Establish the interview session.
7. Enable the required interview features such as communication and coding.

Code Execution Algorithm

The code execution process receives the candidate's program and selected programming language and processes it to produce the execution result.

Steps:

1. Accept the source code and programming language.
2. Validate the submitted code.
3. Receive the required input, if any.
4. Execute the program using the appropriate execution environment.
5. Capture the program output or error.
6. Display the execution result to the user.
7. Store relevant execution details for the assessment.

AI-Assisted Code Evaluation Algorithm

This process uses an AI service to analyse the candidate's submitted solution and generate useful feedback.

Steps:

1. Collect the submitted source code.
2. Send the code and relevant question details to the AI evaluation service.

3. Analyse the solution for correctness, quality and complexity.
4. Generate observations and improvement suggestions.
5. Return the evaluation result.
6. Display the result to the Interviewer for consideration.

Proctoring Algorithm

The proctoring process monitors selected browser activities during an interview and records suspicious events.

Steps:

1. Start monitoring when the interview begins.
2. Detect events such as tab switching or window minimisation.
3. Record the detected event with relevant session information.
4. Display a warning to the Candidate when required.
5. Notify or make the incident available to the Interviewer.
6. Continue monitoring throughout the session.
7. Apply the configured session rule if the permitted incident limit is exceeded.

These algorithms work together to support the main operations of Talent Flow and provide a structured process for conducting, monitoring and evaluating remote technical interviews.

5.3 Architecture

The architecture of Talent Flow is organised into four logical layers: the input layer, the processing layer, the database and external services layer, and the output layer, as shown in Figure 5.3. The interviewer and the candidate supply input data; a set of cooperating processes handle authentication, interview management, scheduling, the real-time interview, evaluation of code and answers, video monitoring and proctoring, and report generation; MongoDB and the Groq API serve as the data store and the external intelligence; and the final results are returned to the two users.

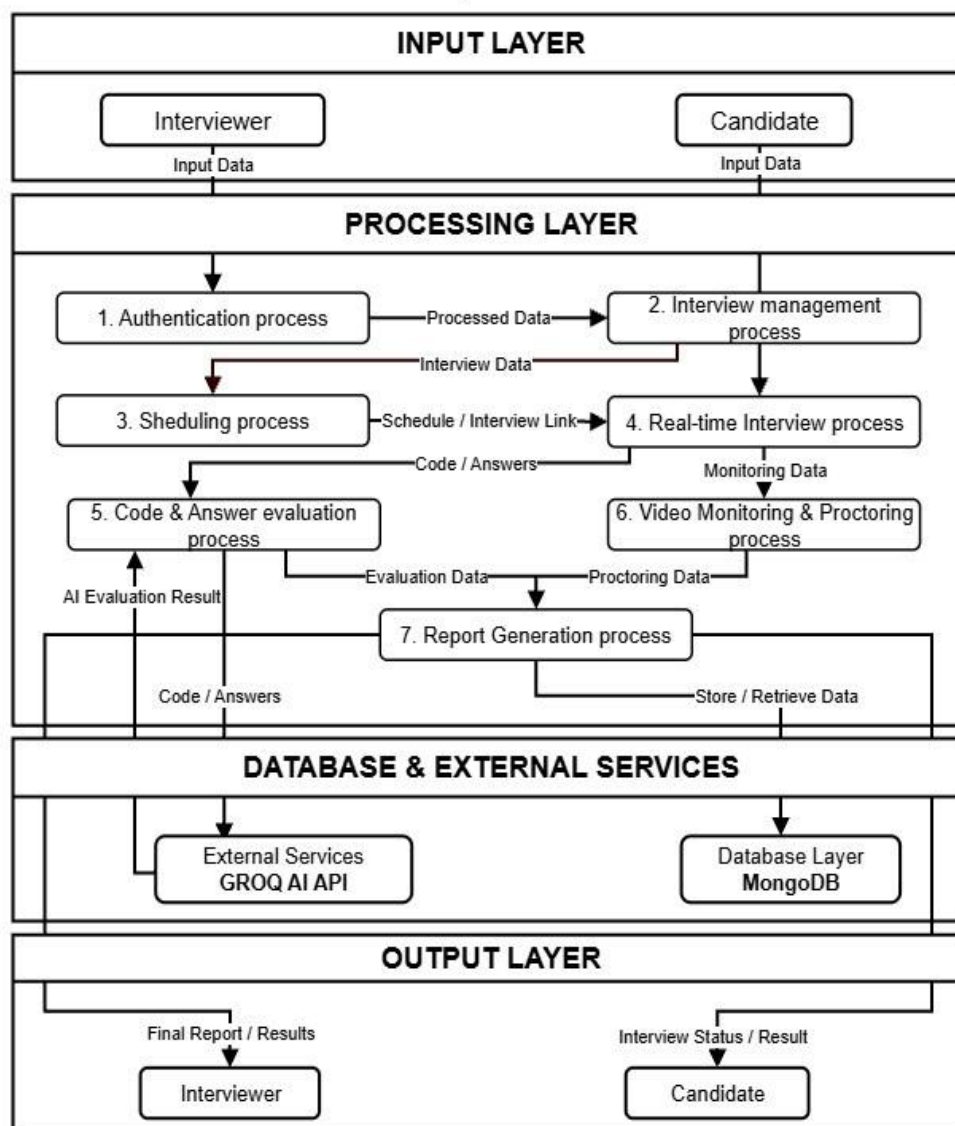


Figure 5.3.1: System architecture of Talent Flow

Input Layer

The input layer contains the two external actors. The **interviewer** supplies interview details, schedules, questions, feedback and commands such as starting the meeting or requesting an AI review. The **candidate** supplies registration and login data, the meeting code, code and answers, and the media streams from the camera and microphone. All input reaches the application through the browser, either as HTTPS requests to the REST API or as messages on the Socket.io connection.

Processing Layer

The processing layer consists of seven cooperating processes:

1. **Authentication process:** validates credentials, creates users and issues JWTs.
2. **Interview management process:** creates interviews, generates meeting codes and manages the interview status.
3. **Scheduling process:** stores date, time, timezone and meeting link and produces the invitation.
4. **Real-time interview process:** manages rooms, signalling, video and audio, code synchronisation and questions.
5. **Code and answer evaluation process:** executes code and requests the AI evaluation, and stores the result.
6. **Video monitoring and proctoring process:** detects violations, maintains the alert counter, records incidents and triggers ejection.
7. **Report generation process:** aggregates all data, computes the final score and produces the report.

The processes exchange processed data, interview data, schedule and interview-link information, code and answers, monitoring data, evaluation data and proctoring data as shown in the architecture figure.

Database and External Services

MongoDB stores all persistent data and is accessed only by the application layer. The Groq API is the only external service. The evaluation process sends the code and question to the API and receives the AI evaluation result. Keeping the database and the API key behind the server ensures that neither can be accessed directly from a browser.

Output Layer

The output layer returns the final report and results to the interviewer and the interview status and results to the candidate. Outputs during the session include the synchronised code, the console output, the AI review, questions and warnings. After the session the output is the PDF scorecard.

Technology Stack Architecture

Figure 5.3.2 maps the logical architecture onto the technologies used. The client layer runs entirely in the browser and hosts the React interface, the Monaco Editor, the WebRTC component, the proctoring listeners and the PDF generator. The communication layer combines REST over HTTPS, Socket.io over WebSocket and peer-to-peer WebRTC

media. The application layer, written in Node.js and Express.js, contains services for authentication, interviews, questions, code execution, AI evaluation, proctoring and reporting, and the data layer consists of MongoDB together with the external Groq service.

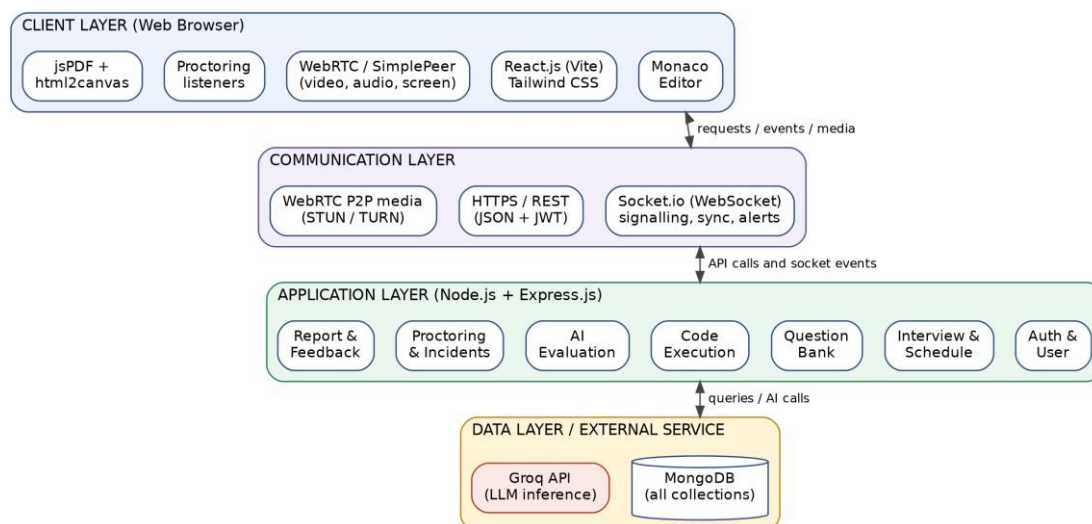


Figure 5.3.2: Layered technology architecture of Talent Flow

5.4 Case Study

Talent Flow can be illustrated through a typical remote technical interview conducted between an Interviewer and a Candidate. Consider a situation where a software company needs to evaluate a candidate for a technical developer position. The Interviewer creates an interview session through Talent Flow and provides the required interview details, including the date, time and assessment information. The Candidate receives the invitation and joins the session through the provided details.

Once the interview begins, the Interviewer and Candidate can communicate through the integrated audio and video facilities. The Interviewer can introduce the assessment and provide technical questions from the question bank. When a programming task is given, the Candidate uses the collaborative coding environment to write the solution. The Interviewer can observe the coding process while the Candidate works on the problem. The code can then be executed with suitable input to check the output and identify possible errors.

During the coding assessment, Talent Flow can also provide AI-assisted analysis of the submitted solution. The generated observations can help the Interviewer understand aspects such as code quality, complexity and possible improvements. At the same time, the proctoring mechanism monitors selected activities such as switching away from the interview tab or minimising the interview window. Detected incidents are recorded so that the Interviewer can review them as part of the assessment.

After completing the technical discussion and coding tasks, the Interviewer records the Candidate's performance using the assessment and feedback features. Ratings, comments, strengths and areas for improvement can be entered into the system. The collected information, including interview details, coding results, AI observations, feedback and relevant proctoring records, can then be combined into a final assessment report.

This case study demonstrates how Talent Flow supports the complete interview process within a single platform, from scheduling and conducting the interview to evaluating the candidate and documenting the final result.

CHAPTER 6

IMPLEMENTATION DETAILS

6.1 Implementation Procedures

The implementation of Talent Flow was carried out in a series of stages to ensure that the different components of the system could be developed and integrated in an organised manner. The process began with analysing the requirements of a remote technical interview platform and identifying the main functions required for Interviewers and Candidates. Based on these requirements, the system structure, user roles, database requirements and major application modules were planned before development.

The frontend of the system was developed to provide a simple interface through which users can register, log in, manage interviews and participate in live assessment sessions. Separate functionalities were provided for Interviewers and Candidates according to their respective roles. The backend was implemented to manage authentication, interview information, questions, sessions, assessments and other application data. A database was used to store and retrieve the required information securely.

After the basic user and interview management features were implemented, the real-time interview components were integrated. Audio and video communication were added to support interaction between the Interviewer and Candidate. The collaborative coding module was then connected with the live interview session so that candidates could write and share code while the Interviewer could monitor the coding activity. The code execution feature was implemented to process submitted programs and display their output or execution errors.

The AI-assisted evaluation module was integrated to analyse candidate code and provide additional observations regarding the solution. Proctoring functionality was also implemented to detect selected activities such as tab switching and window minimisation during an interview. Relevant incidents were recorded and made available for review by the Interviewer.

Finally, the assessment and reporting features were integrated with the completed modules. Interviewers can record ratings, comments, strengths and improvement areas after the session. The collected information is used to prepare the final assessment report. Each major module was tested during development, and integration testing was performed to

verify that the different components worked together correctly. This step-by-step implementation process helped build Talent Flow as a complete platform for conducting and managing remote technical interviews.

6.2 Testing Methods and Results

Testing was carried out throughout the development of Talent Flow to verify that the different modules function correctly and work together as expected. The testing process used sample data representing the activities normally performed during a technical interview. Both valid and invalid inputs were included to check the system under different conditions.

The testing dataset was prepared specifically for the Talent Flow application and consisted of sample records required for different modules. It included **user data** containing Interviewer and Candidate accounts, **interview data** containing session details and schedules, and **question data** containing technical questions with different difficulty levels. The coding dataset contained sample programming problems, source codes, custom inputs and expected outputs. It included correct programs, programs containing logical errors, syntax errors and different input conditions. **Assessment data** included sample ratings, comments, strengths and improvement areas. For testing the proctoring module, simulated events such as tab switching and window minimisation were generated to verify incident detection and recording. These datasets were used only for testing and did not represent real candidate information.

The main testing strategies included **unit testing, integration testing, functional testing and system testing**. Unit testing was used to verify individual functions such as authentication, interview creation, question management, code submission and feedback entry. Integration testing checked whether different modules exchanged data correctly. Functional testing verified that each feature produced the expected result for given inputs, while system testing evaluated the complete application workflow.

Testing Results

Module / Feature	Test Performed	Result
User Authentication	Valid and invalid login details	Passed
Interview Management	Create, schedule and join interview	Passed
Video Communication	Audio/video connection between users	Passed
Collaborative Coding	Shared code editing and updates	Passed
Code Execution	Correct code, errors and custom inputs	Passed
AI Code Evaluation	Analysis of sample code solutions	Passed
Proctoring	Tab switching and window minimisation	Passed
Assessment	Ratings, comments and feedback	Passed
Report Generation	Generate final assessment report	Passed

The coding module was tested using the prepared programming dataset to verify successful execution, incorrect outputs, syntax errors and different input values. The AI-assisted evaluation feature was tested with different code samples to check whether useful observations and improvement suggestions were generated. Proctoring was tested using simulated browser activities to verify that incidents were detected and recorded correctly.

The real-time interview features were tested using separate Interviewer and Candidate sessions. Audio/video communication, collaborative coding, interview scheduling and session joining were checked under different conditions. The assessment and reporting modules were tested by entering sample evaluation data and verifying the generated report.

The testing results showed that the major functions of Talent Flow worked according to the expected requirements. Issues identified during testing were corrected and the affected modules were tested again to confirm proper operation.

CHAPTER 7

ADVANTAGES AND DISADVANTAGES

7.1 Advantages

Talent Flow provides several advantages by bringing the major activities of a remote technical interview into a single platform. It reduces the need to switch between separate applications and provides a more organised environment for both Interviewers and Candidates.

- **Unified Interview Environment:** Communication, coding, assessment and reporting are handled within the same platform.
- **Real-Time Collaboration:** Interviewers and Candidates can communicate and work together during the technical assessment.
- **Efficient Coding Assessment:** Candidates can write and execute code during the interview without depending on an external coding platform.
- **AI-Assisted Evaluation:** AI-based analysis provides additional observations about submitted code, helping the Interviewer during the evaluation process.
- **Improved Interview Monitoring:** Proctoring features help identify and record selected activities such as tab switching and window minimisation.
- **Structured Evaluation:** Interviewers can record ratings, comments, strengths and areas for improvement in an organised manner.
- **Easy Interview Management:** Scheduling, question management and interview sessions can be handled through the same system.
- **Centralised Records:** Interview-related information, including coding results, feedback and monitoring records, can be maintained together.
- **Report Generation:** The collected assessment information can be organised into a final report, reducing the effort required for manual documentation.
- **Convenient Remote Access:** Since the platform is web-based, users can participate in interviews through a supported browser without requiring a separate desktop application.

Overall, these advantages make Talent Flow a practical platform for managing remote technical interviews in a more organised manner. By combining communication, coding, evaluation, monitoring and reporting, the system can simplify the interview workflow and reduce the effort involved in managing different stages of the assessment.

7.2 Disadvantages

Although Talent Flow provides an integrated environment for remote technical interviews, the system has some limitations. These limitations are mainly related to internet connectivity, system resources, external services and the nature of remote assessment.

- **Internet Dependency:** Stable internet connectivity is required for smooth audio/video communication, collaborative coding and other real-time features.
- **Device Requirements:** Users need a suitable computer or laptop with a working camera, microphone and supported web browser for a complete interview experience.
- **External Service Dependency:** Some functions, such as AI-assisted code evaluation, may depend on external APIs and can be affected by service availability or usage limitations.
- **Code Execution Risks:** Running programs during an interview requires a controlled execution environment to prevent unsafe or resource-intensive code from affecting the system.
- **Browser Limitations:** Proctoring features based on browser activity may not detect every possible method of avoiding or bypassing monitoring.
- **AI Evaluation Limitations:** AI-generated code analysis may not always provide completely accurate results and should therefore be treated as supporting information rather than a replacement for the Interviewer's judgement.
- **Real-Time Performance Issues:** Network delays or temporary server problems may affect video communication, collaborative editing or code execution during an interview.
- **Initial Learning Requirement:** Users may need some time to become familiar with the different features of the platform before conducting or participating in an interview.

Overall, these limitations do not prevent the use of Talent Flow, but they indicate areas that require attention during deployment and future development. Improving network handling, security, execution isolation, monitoring capabilities and external service reliability can further strengthen the system.

CHAPTER 8

RESULT AND DISCUSSION

8.1 Analysis

The analysis of Talent Flow was carried out based on the implementation and testing of its major modules. The results indicate that the developed system can support the main activities involved in a remote technical interview within a single web-based environment. The core functions, including user authentication, interview management, real-time communication, collaborative coding, code execution, AI-assisted evaluation, proctoring, assessment and report generation, were tested using suitable sample data and different input conditions.

The testing showed that Interviewers and Candidates could access the system according to their assigned roles and participate in interview sessions successfully. The communication and collaborative coding features supported interaction during the technical assessment, while the code execution module allowed submitted programs to be tested with different inputs. The AI-assisted evaluation module provided additional observations about submitted solutions, which can support the Interviewer during the assessment process.

The analysis also showed that the proctoring module was able to detect selected activities such as tab switching and window minimisation and record the corresponding incidents. The assessment module allowed Interviewers to enter ratings and feedback in an organised manner, while the reporting feature helped combine relevant interview information into a final assessment document.

Another important observation was the coordination between the different modules during the complete interview workflow. Information entered or generated at one stage could be used by the related features in later stages, such as using interview details for session management and assessment information for report generation. This integration helped maintain the interview data in a structured manner and reduced unnecessary duplication of information across different stages of the process.

Overall, the results demonstrate that the major requirements identified for Talent Flow were addressed by the implemented system. At the same time, the analysis highlights certain practical limitations, such as dependence on stable internet connectivity, browser capabilities and external services. These factors can influence the performance of some

features during actual use. The results therefore indicate that Talent Flow provides a functional foundation for conducting remote technical interviews while also identifying areas that can be improved in future versions.

The key observations obtained from the analysis of the system include:

- **Successful Role Management:** Interviewers and Candidates were able to access the functions available to their respective roles.
- **Integrated Interview Process:** The major interview activities could be performed within the same platform, reducing the need to use multiple separate applications.
- **Effective Real-Time Interaction:** Audio/video communication and collaborative coding supported interaction between the Interviewer and Candidate during the interview session.
- **Reliable Code Testing:** The code execution module handled different test conditions, including valid programs, syntax errors and incorrect inputs.
- **AI-Assisted Evaluation:** The AI evaluation feature generated additional observations about submitted code, which can support the Interviewer during assessment.
- **Proctoring Detection:** Selected activities such as tab switching and window minimisation were detected and recorded as interview incidents.
- **Structured Assessment:** Interviewers could record ratings, comments, strengths and areas for improvement in an organised manner.
- **Report Generation:** The system successfully combined relevant assessment information into a final report.
- **Module Integration:** The tested modules worked together to support the overall workflow from interview creation and participation to assessment and reporting.
- **Identified Limitations:** The analysis also identified practical dependencies such as stable internet connectivity, browser capabilities and availability of external services.

Overall, the observations indicate that Talent Flow achieved the major functional requirements defined for the proposed remote technical interview platform.

CHAPTER 9

CONCLUSION

9.1 Summary

Talent Flow is a web-based platform developed to simplify and organise the process of conducting remote technical interviews. The system brings important interview activities such as communication, collaborative coding, code execution, question management, evaluation, monitoring and report generation into a single platform. This reduces the need to depend on multiple separate applications during an interview.

The platform provides separate roles for Interviewers and Candidates. Interviewers can create questions, manage interviews, conduct technical assessments, monitor interview sessions and provide feedback. Candidates can join interviews, communicate with the Interviewer, solve coding problems and submit their solutions. Real-time communication and collaborative coding features support better interaction during the technical assessment. Talent Flow also includes AI-assisted code evaluation, which provides additional observations about submitted solutions. The proctoring module monitors selected activities such as tab switching and window minimisation and records detected incidents. At the end of the interview, assessment details and other relevant information can be organised into a final report.

The development and testing of the system showed that the major modules can work together as part of a complete remote technical interview workflow. Overall, Talent Flow provides a structured approach for managing technical interviews and can reduce the manual effort involved in conducting and documenting remote assessments.

9.2 Limitations

Although Talent Flow provides the required features for conducting remote technical interviews, the system has some practical limitations.

- **Internet Dependency:** Real-time video communication, collaborative coding and other online features require a stable internet connection.
- **Device Requirements:** The quality of the interview may depend on the candidate's and Interviewer's computer, camera, microphone and other hardware.
- **External Service Dependency:** Some functions depend on external services or APIs, which may affect availability and performance.

- **AI Evaluation Limitations:** AI-generated code analysis may not always understand every programming approach or provide completely accurate feedback.
- **Browser Limitations:** Some monitoring and real-time features may behave differently depending on the browser and its permissions.
- **Code Execution Risks:** Executing submitted programs requires proper security controls to prevent unsafe code from affecting the system.
- **Performance Issues:** A large number of simultaneous interview sessions may require additional server resources and performance optimisation.
- **Initial Learning Requirement:** New users may need some time to become familiar with the different features of the platform.

These limitations provide areas for further improvement and optimisation as the system is developed into a larger-scale platform.

9.3 Future Scope

Talent Flow can be further improved by adding advanced features and improving the existing modules. The future development of the system can focus on scalability, security, automation and a better user experience.

Possible future enhancements include:

- **Advanced AI Evaluation:** Improve AI-based analysis to evaluate code quality, efficiency, programming practices and problem-solving approaches in greater detail.
- **Enhanced Proctoring:** Introduce additional monitoring mechanisms such as face presence detection, multiple-person detection and suspicious activity analysis, while considering privacy requirements.
- **Scalable Infrastructure:** Deploy the system using cloud infrastructure to support a larger number of simultaneous interviews.
- **More Programming Languages:** Expand the code execution environment to support additional programming languages and frameworks.
- **Advanced Analytics:** Provide dashboards showing candidate performance, assessment trends and interview statistics.
- **Improved Security:** Add stronger isolation and security mechanisms for code execution, authentication and stored interview data.

- **Automated Scheduling:** Integrate calendar services and automated notifications for scheduled interviews.
- **Mobile Support:** Develop responsive or dedicated mobile support for selected interview management activities.
- **Enhanced Reports:** Provide more detailed and customisable assessment reports for Interviewers and organisations.

These improvements can help Talent Flow evolve from a project-level platform into a more comprehensive system for remote technical recruitment and assessment.

9.4 Applications

Talent Flow can be applied in different situations where technical skills need to be assessed remotely. Its combination of communication, coding, evaluation and monitoring features makes it suitable for several use cases.

- **Technical Recruitment:** Companies can use the platform to conduct coding interviews and evaluate software development candidates remotely.
- **Campus Recruitment:** Educational institutions and recruitment teams can use Talent Flow for conducting technical placement interviews without requiring candidates to be physically present.
- **Online Assessments:** Organisations can conduct programming-based assessments with code execution and structured evaluation.
- **Training and Skill Evaluation:** Training institutions can use the system to evaluate learners' programming and problem-solving abilities.
- **Remote Hiring:** Distributed organisations can conduct interviews with candidates located in different cities or regions.
- **Technical Certification:** The platform can support practical programming assessments as part of technical certification or skill-verification processes.
- **Internal Employee Assessment:** Organisations can use the system for evaluating employees during technical training, skill assessments or internal role changes.

Overall, Talent Flow demonstrates how multiple technologies can be combined to create a structured and convenient environment for remote technical interviews. The project provides a foundation that can be extended with stronger security, improved AI capabilities, enhanced analytics and scalable infrastructure in future versions.

CHAPTER 10

APPENDICES

10.1 Sample Code

```

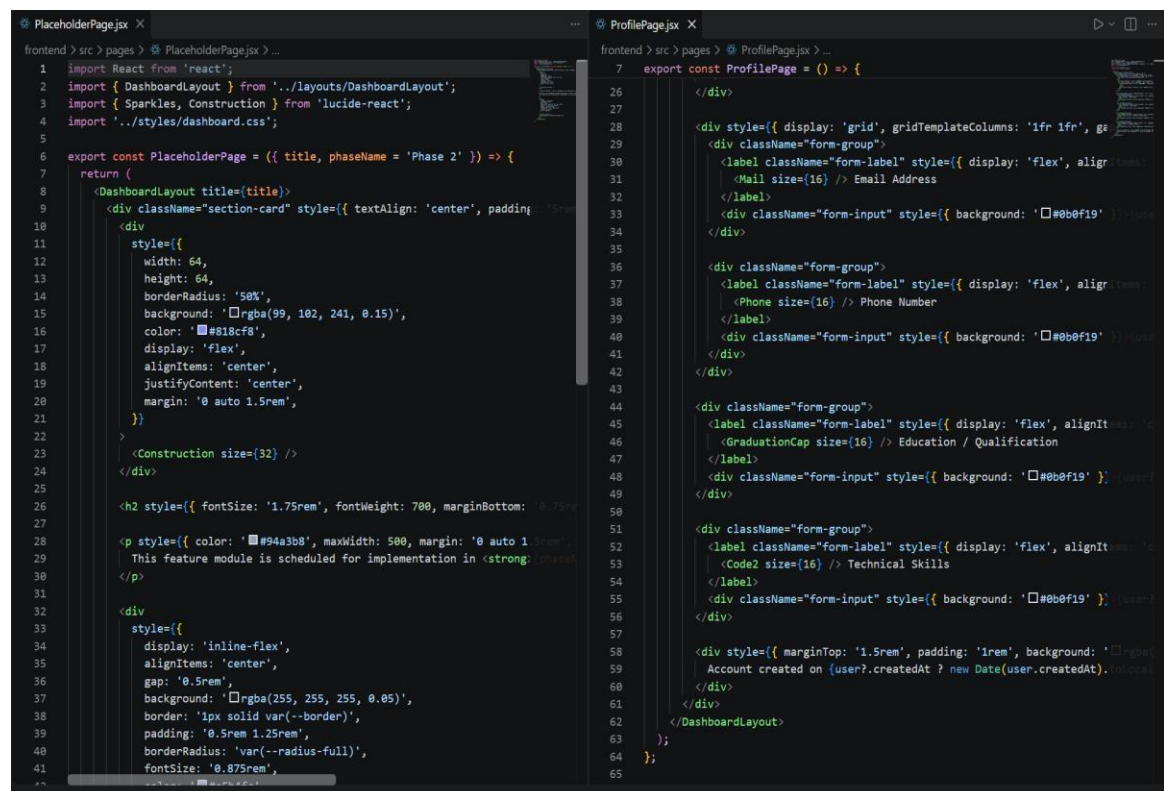
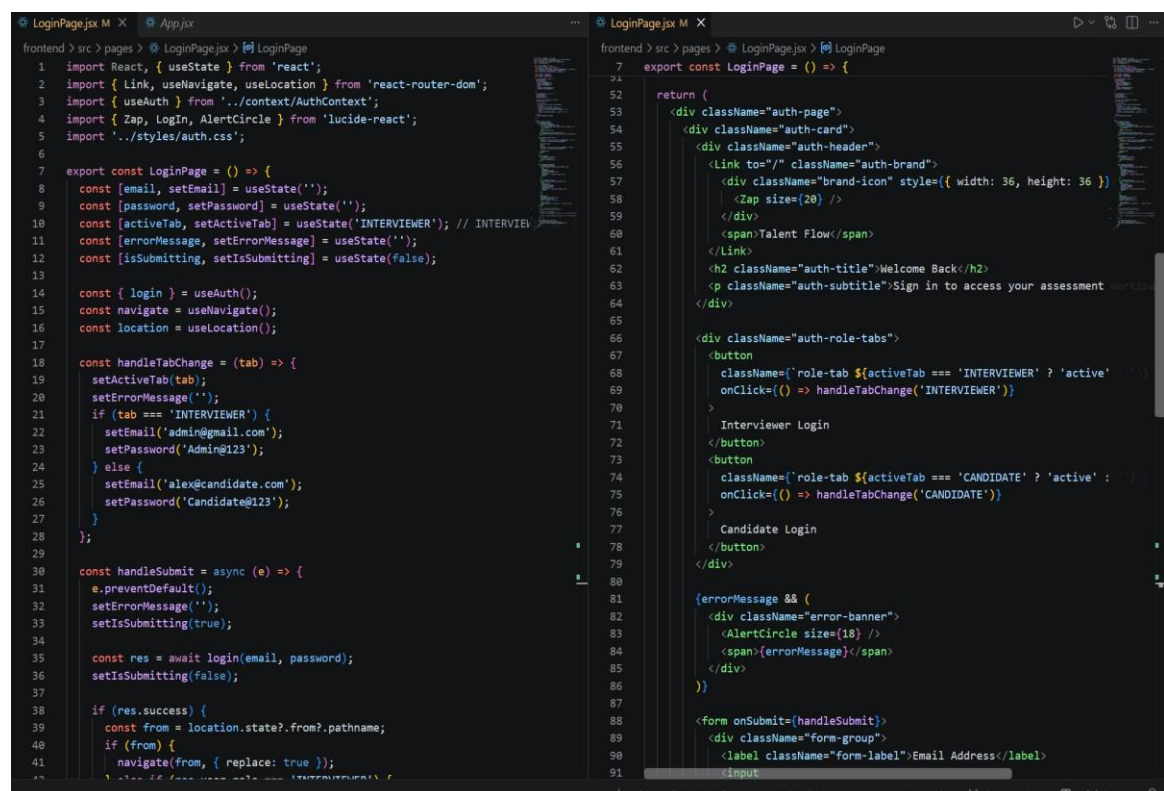
server.js X
backend > src > server.js > ...
1 require('dotenv').config();
2 const express = require('express');
3 const http = require('http');
4 const { Server } = require('socket.io');
5 const cors = require('cors');
6 const helmet = require('helmet');
7 const rateLimit = require('express-rate-limit');
8
9 const authRoutes = require('./routes/authRoutes');
10 const dashboardRoutes = require('./routes/dashboardRoutes');
11 const candidateRoutes = require('./routes/candidateRoutes');
12 const questionRoutes = require('./routes/questionRoutes');
13 const assessmentRoutes = require('./routes/assessmentRoutes');
14 const sessionRoutes = require('./routes/sessionRoutes');
15 const executionRoutes = require('./routes/executionRoutes');
16 const submissionRoutes = require('./routes/submissionRoutes');
17 const proctoringRoutes = require('./routes/proctoringRoutes');
18 const reportRoutes = require('./routes/reportRoutes');
19 const feedbackRoutes = require('./routes/feedbackRoutes');
20 const { notFoundHandler, errorHandler } = require('./middleware/errorHandler');
21 const { initSockets } = require('./sockets');
22
23 const app = express();
24 const server = http.createServer(app);
25
26 // Security Middleware
27 app.use(helmet());
28
29 // CORS Configuration
30 const allowedOrigin = process.env.CLIENT_URL || 'http://localhost:5173';
31 app.use(
32   cors({
33     origin: (origin, callback) => callback(null, true),
34     credentials: true,
35     methods: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'OPTIONS'],
36     allowedHeaders: ['Content-Type', 'Authorization'],
37   })
38 );
39
40 // Body Parsing Middleware
41 app.use(express.json());
42 app.use(express.urlencoded({ extended: true }));
43
44 // Rate Limiting
45 const limiter = rateLimit({
46   windowMs: 15 * 60 * 1000, // 15 minutes
47   max: 300, // limit each IP to 300 requests per window
48   standardHeaders: true,
49   legacyHeaders: false,
50   message: {
51     success: false,
52   },
53 });
54
55 const limiter = rateLimit({
56   windowMs: 15 * 60 * 1000, // 15 minutes
57   max: 300, // limit each IP to 300 requests per window
58   standardHeaders: true,
59   legacyHeaders: false,
60   message: {
61     success: false,
62   },
63 });
64
65 // Socket.io Initialization
66 const io = new Server(server, {
67   cors: {
68     origin: (origin, callback) => callback(null, true),
69     credentials: true,
70     methods: ['GET', 'POST'],
71   },
72 });
73
74 // Health check endpoint
75 app.get('/api/health', (req, res) => {
76   res.status(200).json({
77     success: true,
78     message: 'Talent Flow API server is healthy',
79     timestamp: new Date().toISOString(),
80   });
81 });
82
83 // Mount Routes
84 app.use('/api/auth', authRoutes);
85 app.use('/api/dashboard', dashboardRoutes);
86 app.use('/api/candidates', candidateRoutes);
87 app.use('/api/questions', questionRoutes);
88 app.use('/api/assessments', assessmentRoutes);
89 app.use('/api/session', sessionRoutes);
90 app.use('/api/code', executionRoutes);
91 app.use('/api/submissions', submissionRoutes);
92 app.use('/api/proctoring', proctoringRoutes);
93 app.use('/api/reports', reportRoutes);
94 app.use('/api/feedback', feedbackRoutes);
95
96 // Error Handling Middleware
97 app.use(notFoundHandler);
98 app.use(errorHandler);
99
100 const PORT = process.env.PORT || 5000;
101
102 server.listen(PORT, () => {
103   console.log('Talent Flow Backend Server running on port ${PORT}');
104   console.log('Configured CORS origin: ${allowedOrigin}');
105 });

```

```

App.jsx X
frontend > src > App.jsx > ...
1 import React from 'react';
2 import { Routes, Route, Navigate } from 'react-router-dom';
3 import { ProtectedRoute } from './components/ProtectedRoute';
4 import { RoleBasedRoute } from './components/RoleBasedRoute';
5
6 import { LandingPage } from './pages/LandingPage';
7 import { LoginPage } from './pages/LoginPage';
8 import { RegisterPage } from './pages/RegisterPage';
9 import { InterviewerDashboard } from './pages/InterviewerDashboard';
10 import { CandidateDashboard } from './pages/CandidateDashboard';
11 import { ProfilePage } from './pages/ProfilePage';
12 import { PlaceholderPage } from './pages/PlaceholderPage';
13
14 // Candidate Management Pages
15 import { CandidateListPage } from './pages/CandidateListPage';
16 import { CandidateFormPage } from './pages/CandidateFormPage';
17 import { CandidateDetailPage } from './pages/CandidateDetailPage';
18
19 // Question Bank Pages
20 import { QuestionListPage } from './pages/QuestionListPage';
21 import { QuestionFormPage } from './pages/QuestionFormPage';
22
23 // Assessment Management Pages
24 import { AssessmentListPage } from './pages/AssessmentListPage';
25 import { AssessmentFormPage } from './pages/AssessmentFormPage';
26 import { AssessmentDetailPage } from './pages/AssessmentDetailPage';
27 import { CandidateAssessmentDetailPage } from './pages/CandidateAssessmentDetailPage';
28
29 // Live Assessment & Phase 5 Reports & Analytics Pages
30 import { InterviewerLiveDashboardPage } from './pages/InterviewerLiveDashboardPage';
31 import { CandidateLiveSessionPage } from './pages/CandidateLiveSessionPage';
32 import { InterviewerProctoringPage } from './pages/InterviewerProctoringPage';
33 import { InterviewerReportListPage } from './pages/InterviewerReportListPage';
34 import { CandidateReportDetailPage } from './pages/CandidateReportDetailPage';
35 import { AssessmentResultsPage } from './pages/AssessmentResultsPage';
36 import { AssessmentAnalyticsPage } from './pages/AssessmentAnalyticsPage';
37
38 import './styles/global.css';
39
40 export default function App() {
41   return (
42     <div>
43       <Routes>
44         {/* Public Routes */}
45         <Route path="/" element={<LandingPage />} />
46         <Route path="/login" element={<LoginPage />} />
47         <Route path="/register" element={<RegisterPage />} />
48       </Routes>
49
50       {/* Protected Interviewer Routes */}
51       <Route
52         path="/interviewer/dashboard"
53         element={
54           <ProtectedRoute>
55             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
56               <InterviewerDashboard />
57             </RoleBasedRoute>
58           </ProtectedRoute>
59         }
60       />
61
62       <Route
63         path="/interviewer/candidates"
64         element={
65           <ProtectedRoute>
66             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
67               <CandidateListPage />
68             </RoleBasedRoute>
69           </ProtectedRoute>
70         }
71       />
72
73       <Route
74         path="/interviewer/candidates/new"
75         element={
76           <ProtectedRoute>
77             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
78               <CandidateFormPage />
79             </RoleBasedRoute>
80           </ProtectedRoute>
81         }
82       />
83
84       <Route
85         path="/interviewer/candidates/:id"
86         element={
87           <ProtectedRoute>
88             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
89               <CandidateDetailPage />
90             </RoleBasedRoute>
91           </ProtectedRoute>
92         }
93       />
94
95       <Route
96         path="/interviewer/candidates/:id/report"
97         element={
98           <ProtectedRoute>
99             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
100               <CandidateReportDetailPage />
101             </RoleBasedRoute>
102           </ProtectedRoute>
103         }
104       />
105
106       <Route
107         path="/interviewer/candidates/:id/session"
108         element={
109           <ProtectedRoute>
110             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
111               <CandidateLiveSessionPage />
112             </RoleBasedRoute>
113           </ProtectedRoute>
114         }
115       />
116
117       <Route
118         path="/interviewer/candidates/:id/proctoring"
119         element={
120           <ProtectedRoute>
121             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
122               <InterviewerProctoringPage />
123             </RoleBasedRoute>
124           </ProtectedRoute>
125         }
126       />
127
128       <Route
129         path="/interviewer/candidates/:id/reports"
130         element={
131           <ProtectedRoute>
132             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
133               <InterviewerReportListPage />
134             </RoleBasedRoute>
135           </ProtectedRoute>
136         }
137       />
138
139       <Route
140         path="/interviewer/candidates/:id/results"
141         element={
142           <ProtectedRoute>
143             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
144               <AssessmentResultsPage />
145             </RoleBasedRoute>
146           </ProtectedRoute>
147         }
148       />
149
150       <Route
151         path="/interviewer/candidates/:id/analytics"
152         element={
153           <ProtectedRoute>
154             <RoleBasedRoute allowedRoles={['INTERVIEWER']}>
155               <AssessmentAnalyticsPage />
156             </RoleBasedRoute>
157           </ProtectedRoute>
158         }
159       />
160     </div>
161   );
162 }

```



Hosted Project Link :

Project Repository: https://github.com/AromalAnill/Talent_Flow

REFERENCES

- [1] Prasath, T. M., et al. (2026). *Intelligent Candidate Monitoring and Interview Surveillance System*. International Journal for Research in Applied Science and Engineering Technology (IJRASET).
- [2] Harshavardhan, et al. (2026). *Secure Real-Time Coding Interview System with AI-Based Evaluation and Monitoring*. International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE).
- [3] *An Intelligent Online Interview System Based on AI*. IEEE.
- [4] *AI-Based Automatic Interview Assessment*. IEEE.
- [5] *Online Remote Proctoring System Using AI*. IEEE.
- [6] *Collaborative Online Code Editor Using WebRTC and Socket.IO*. IEEE.
- [7] *Remote Technical Interview Platform with Real-Time Collaboration*. IEEE.
- [8] *CodeSync: Real-Time Collaborative Code Editor*. (2026). IJCRT.
- [9] *REALCODEAI: Real-Time Code Collaboration and Interview Platform*. (2025). IJSRSET.
- [10] *Code Together: Collaborative Code Editing Using CRDT*. (2024).
- [11] *Smart AI-Based Online Proctoring System*. (2022). IEEE.
- [12] *AI-Based Fully Automated Online Proctoring Systems: A Survey*. (2022). IEEE.
- [13] *Deep Learning Based Online Exam Proctoring Systems: A Review*. (2022). IJSRSET.
- [14] World Wide Web Consortium (2021). *WebRTC 1.0: Real-Time Communication Between Browsers*. W3C Recommendation. <https://www.w3.org/TR/webrtc/>
- [15] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, IETF. <https://www.rfc-editor.org/rfc/rfc7519>
- [16] Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol*. RFC 6455, IETF. <https://www.rfc-editor.org/rfc/rfc6455>
- [17] World Wide Web Consortium. *Page Visibility (Second Edition)*. W3C Recommendation. <https://www.w3.org/TR/page-visibility/>
- [18] Meta Platforms, Inc. *React Documentation*. <https://react.dev>
- [19] OpenJS Foundation. *Node.js Documentation*. <https://nodejs.org/docs>

- [20] Express.js. *Express Documentation*. <https://expressjs.com>
- [21] MongoDB, Inc. *MongoDB Manual*. <https://www.mongodb.com/docs/>
- [22] Socket.IO. *Socket.IO Documentation*. <https://socket.io/docs/>
- [23] Microsoft. *Monaco Editor*. <https://microsoft.github.io/monaco-editor/>
- [24] Groq, Inc. *GroqCloud API Documentation*. <https://console.groq.com/docs>
- [25] Aboukhadijeh, F. *simple-peer: Simple WebRTC video, voice and data channels*. <https://github.com/feross/simple-peer>
- [26] jsPDF contributors. *jsPDF: Client-side JavaScript PDF generation*. <https://github.com/parallax/jsPDF>
- [27] Niklas von Herten. *html2canvas*. <https://html2canvas.herten.com>
- [28] Tailwind Labs. *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>
- [29] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.
- [30] Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.

Aromal Anil

Talent_Flow_MCA_Mini_Project_Report_Final

 test

Document Details

Submission ID**trn:oid::10159:153455072****Submission Date****Oct 1, 2026, 10:08 PM GMT+5:30****Download Date****Oct 1, 2026, 10:19 PM GMT+5:30****File Name****Talent_Flow_MCA_Mini_Project_Report_Final.pdf****File Size****3.1 MB****60 Pages****13,306 Words****79,745 Characters**

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Exclusions

- 66 Excluded Matches

Match Groups

-  **65** Not Cited or Quoted 5%
Matches with neither in-text citation nor quotation marks
-  **0** Missing Quotations 0%
Matches that are still very similar to source material
-  **1** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
-  **0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 1%  Publications
- 4%  Submitted works (Student Papers)